

Fast Algorithms for Bin Packing*

DAVID S. JOHNSON[†]

Project MAC, Massachusetts Institute of Technology, Cambridge, Massachusetts 02139

Low-order polynomial time algorithms for near-optimal solutions to the problem of bin packing are studied. The previously analyzed FIRST FIT and BEST FIT packing rules are shown to be members of a more generalized class of packing rules all of which have the same worst case behavior. If the input list is in decreasing order, the worst case behavior of the packing rules in the class is considerably improved and, if not the same for all, at least restricted to a narrow range of possibilities. Finally, after showing that any implementation of a packing rule in the class requires at least $O(n \log n)$ comparisons, we present linear-time approximations to these packing rules whose worst case behavior is as good as that of FIRST FIT under a large variety of restrictions on the input.

1. INTRODUCTION

The *bin packing* problem has recently received attention [5, 11] as a model for such problems as table forming, packing of tracks on a disk, and prepaging. It also is a simplified form of many "stock cutting" problems encountered in industry [2]. Suppose we are given a finite list $L = \langle a_1, a_2, \dots, a_n \rangle$ of real numbers in the range $(0, 1]$, and a sequence of unit-capacity bins, $\text{BIN}_1, \text{BIN}_2, \dots$, extending from left to right. The problem is to find an assignment or *packing* of the numbers into the bins so that no bin has contents totaling more than one, and yet the number of bins *used*, i.e., nonempty, is minimized. For a given list L we will denote this minimum by L^* .

In this paper we consider simple heuristic algorithms for producing packings which are guaranteed of using no more than a fixed percentage of bins in excess of this minimum number. More formally, if S is an algorithm which generates packings, let $S(L)$ be the number of bins used in the packing resulting when S is applied to list L . We are interested in algorithms for which the worst case bounds on the quantity

* The research reported here was supported in part by Project MAC, an M.I.T. research program sponsored by the Advanced Research Projects Agency, Department of Defense, under Office of Naval Research Contract No. N00014-70-A-0362-0006 and the National Science Foundation under Contract No. GJ00-4327. A preliminary version of this paper was presented at the Thirteenth Annual IEEE Symposium on Switching and Automata Theory.

[†] Current address: Department 1216, Bell Telephone Laboratories, Murray Hill, NJ 07974.

$S(L)/L^*$ are not greatly in excess of 1. To express our results we shall use an asymptotic measure, $R[S]$, defined as follows. Let

$$R[S](k) = \max\{S(L)/L^*: \text{lists } L \text{ such that } L^* = k\}.$$

Then we define

$$R[S] = \limsup R[S](k),$$

where in all the results presented here, the $\limsup$ is actually a limit. In a similar fashion, we can define $R[S, t]$ to be the $\limsup$ for the special case where no number in L is larger than $t \leq 1$. Note that if $t_2 \geq t_1$, a list with no number larger than t_1 is also a list which no number larger than t_2 , and so $R[S, t]$ is an increasing function in t , with maximum value $R[S, 1] = R[S]$.

It has previously been shown [4, 5, 11] that, for the two algorithms FIRST FIT and BEST FIT and $t > 1/2$, we have that

$$R[S, t] = 17/10 \tag{1.1}$$

and, for $m = \lfloor 1/t \rfloor \geq 2$,

$$R[S, t] = 1 + 1/m. \tag{1.2}$$

Moreover, if we put L into decreasing order, largest number first, and *then* apply one of the above algorithms, the combined algorithm obeys

$$11/9 \leq R[S] \leq 5/4. \tag{1.3}$$

In this paper we show that (1.1), (1.2), and (1.3) actually hold for a large class of related algorithms, the ALMOST-ANY FIT ALGORITHMS, and that (1.3) in fact holds for an even wider class, the ANY FIT algorithms.

In Section 2 we define the algorithms and consider the “on-line” case, the use of the algorithms with no preprocessing, extending the applicability of Results (1.1) and (1.2). In Sections 3 and 4 we study the algorithms in the case where the list is first put in decreasing order, and prove the extension of Result (1.3). In Section 5 we discuss the implementation of ANY FIT algorithms. They are shown to all require at least $O(n \log n)$ comparisons, under reasonable assumptions about their implementation, and we show how this bound can be achieved for a number of the particular algorithms we have previously introduced. Finally, we investigate the possibility of reducing costs by using linear time approximations to both the preprocessing and the basic algorithms, and place bounds on the worst case behavior of these new algorithms. The best is GROUP FIT GROUPED, for which $R[S] = 1.5$.

Bin packing, like many other interesting combinatorial problems, is “polynomial complete” in the sense of Cook and Karp [3, 9]. This means that if there exists a polynomial-time algorithm for finding an optimal packing, there will also be a poly-

nomial time algorithm for solving each of the other problems of the class, and vice versa. Since the class contains many problems, such as the traveling salesman problem and tautology testing, which have become notorious for their computational intractability, it appears that they all may well *require* greater than polynomial time for their solution.

The approach taken by this paper allows us to obtain useful algorithms while sidestepping these difficulties, and may well be applicable to other such "hard" problems [6].

2. ANY FIT ALGORITHMS

In what follows, it will be useful to have alternative ways of referring to *lists* and *packings*. A *list* L may be thought of as a sequence of real numbers in $(0, 1]$, or alternatively as the following:

- (1) A finite set $\text{PIECES}(L)$ of *pieces*.
- (2) A 1-1 map $\text{rank}_L: \text{PIECES}(L) \rightarrow \{1, 2, \dots, |\text{PIECES}(L)|\}$.
- (3) A map $\text{size}_L: \text{PIECES}(L) \rightarrow (0, 1]$.

The *rank*, i.e., position, of piece x in list L is given by $\text{rank}_L(x)$, and $\text{size}_L(x)$ gives its *size*. Usually we can drop the subscript L without causing confusion. By $W(L)$ we will mean

$$\sum_{x \in \text{PIECES}(L)} \text{size}(x).$$

This definition corresponds to the earlier notion of a list as a sequence of real numbers, for we may also write $L = \langle a_1, \dots, a_n \rangle$ where $n = |\text{PIECES}(L)|$, and $a_i = \text{size}_L(\text{rank}_L^{-1}(i))$. We shall use the two notations interchangeably, the latter being especially useful for giving examples of lists, such as " $L = \langle 0.01, 0.5, 0.25, 0.01 \rangle$," and for talking about the *concatenation* $L1 \cdot L2$ of two lists $L1$ and $L2$, and a *permutation* L' of a given list L .

The basic notion of a packing P of a list $L = \langle a_1, \dots, a_n \rangle$ implied by our description of the bin packing problem is simply a map

$$P: \{1, 2, \dots, n\} \rightarrow \{1, 2, \dots, n\}.$$

such that for all j , $1 \leq j \leq n$, $\sum_{P(i)=j} a_i \leq 1$.

A second, equivalent, definition, whose additional structure has considerable descriptive power and will be of use in formalizing our arguments, specifies a packing P of a list L in an inverse manner by giving a 1-1 partial map

$$\text{piece}_P: \{(j, h): 1 \leq j \leq n, h \geq 1\} \rightarrow \text{PIECES}(L),$$

obeying the following three *packing properties*:

- (1) For each j , $1 \leq j \leq n = |\text{PIECES}(L)|$,

$$\sum_{(j,h) \in \text{Domain}(\text{piece}_P)} \text{size}(\text{piece}_P(j, h)) \leq 1.$$

- (2) $(j, h) \in \text{Domain}(\text{piece}_P)$ and $h > 1$

$$\Rightarrow (j, h - 1) \in \text{Domain}(\text{piece}_P),$$

- (3) $(j, h), (j, h') \in \text{Domain}(\text{piece}_P)$ and $h > h'$

$$\Rightarrow \text{rank}(\text{piece}_P(j, h)) \geq \text{rank}(\text{piece}_P(j, h')).$$

We say that $\{(j, h): h \geq 1\}$ is the *set of positions* in BIN_j , with $(j, 1)$ the *bottom* position. If $(j, h) \in \text{Domain}(\text{piece}_P)$ then we say that position (j, h) is *filled*, and that $\text{piece}_P(j, h)$ *fills* it and is *contained* in BIN_j . Properties (2) and (3) assert that the pieces contained in BIN_j fill the bottom-most positions, and do so by order of rank. In this definition, we shall think of P as a sequence of bins, each with its positions filled according to piece_P . We shall often display packings by means of pictures, with the bins drawn as upright rectangles, the pieces as smaller rectangles fitted within the bins by order of position, $\text{piece}_P(j, 1)$ on the bottom. See for instance Figs. 1 and 2.

In order to describe the contents of a bin, we introduce the following notation:

$$\text{cont}_P(j) = \{b: b = \text{piece}_P(j, h) \text{ for some } h\},$$

$$\text{height}_P(j) = |\text{cont}_P(j)|,$$

$$\text{level}_P(j) = \sum_{b \in \text{cont}_P(j)} \text{size}(b), \text{ and}$$

$$\text{gap}_P(j) = 1 - \text{level}_P(j).$$

BIN_j is *used* or *nonempty* if $\text{level}_P(j) > 0$, otherwise it is *empty*. The right-most nonempty bin in a packing will be called the last bin.

A *segment* PI of a packing P will be any set of consecutive bins from P . $\#PI$ is the number of nonempty bins in PI . $\text{cont}(PI) = \{x: x \in \text{cont}_P(j) \text{ and } \text{BIN}_j \text{ in } PI\}$. $w(PI) = \sum_{x \in \text{cont}(PI)} \text{size}(x)$. Since P may be considered a segment of itself, all these definitions apply to P as well, and so $\#P$ is the number of bins used in P .

In general a bin packing algorithm is *any* method which, given a list L , produces a packing P of L . However, the algorithms we shall spend most of our time studying all fit into a specific two-part format and will be called *two-part* algorithms. The first part will be a reordering of the list according to the *preprocessing rule* of the algorithm, the second the generation of a packing of the recorded list using the *packing rule* of the algorithm. A two-part algorithm will be specified by giving these two rules, which will be applied in the following manner:

PART 1 (reordering the list via the preprocessing rule): Reorder L (construct a new *rank* function) to obtain a permutation $L' = \langle b_1, b_2, \dots, b_n \rangle$ that obeys the preprocessing rule.

PART 2 (Generation of a packing of L' via the packing rule):

(1) Let P be a sequence of n empty bins. Set $i = 1$. P may be thought of as a packing of the empty list, and in what follows, as a packing of $\langle b_1, \dots, b_{i-1} \rangle$, with $\text{height}_P(j)$ and $\text{level}_P(j)$ defined for P as above. At this initial step we have $\text{height}_P(j) = \text{level}_P(j) = 0$, $1 \leq j \leq n$.

(2) If $i > n$, halt and return P .

(3) Let $b \in \text{PIECES}(L')$ be that piece with rank i . In accord with the packing rule of the algorithm, which bases its decision solely on $\text{size}(b)$ and the current values of $\text{level}_P(j)$, choose a j such that b will fit in BIN_j (such that $\text{level}_P(j) + \text{size}(b) \leq 1$).

(4) Set $P =$ the packing of $\langle b_1, \dots, b_i \rangle$ obtained by putting b in position $(j, \text{height}_P(j) + 1)$ of P and leaving all other pieces in their old positions. Set $i = i + 1$ and go to 2.

Remark. Since we do not specify *how* the list is to be reordered so that it obeys the preprocessing rule, or how a bin which obeys the packing rule is chosen, the above description of the "algorithm" is not complete. However, it is sufficient to specify which packings are generable, and we shall continue to refer to the combination of a preprocessing rule and a packing rule as an algorithm, leaving the question of implementation for Section 5. If the algorithm in question is S , then a packing resulting from the above procedure applied to list L will be called an S -packing of L .

In this section we are concerned with algorithms which can be thought of as operating *on-line* since their preprocessing rule is vacuous, that is, $L' = L$. Such algorithms are specified completely by their packing rules, the ones previously studied being the following..

FIRST FIT (*FF*). Assign b to the leftmost bin into which it will fit. (let $j = \min\{j: \text{level}_P(j) + \text{size}(b) \leq 1\}$.)

BEST FIT (*BF*). Assign b so as to maximize the level of the bin into which it goes. (Let j be that k for which $\text{level}_P(k) + \text{size}(b)$ is closest to, without exceeding, 1). If more than one bin yields this maximum, choose the left-most one.

These two algorithms can be seen to have more in common than just the same worst case bounds. Close inspection reveals that they both satisfy the following two constraints:

(1) ANY FIT CONSTRAINT. If BIN_j is empty, it may not be chosen unless b will not fit in any bin to the left of BIN_j .

This constraint still allows us to assign b to *any* nonempty bin into which it will fit. The second constraint lets us assign b to *almost* any nonempty bin, there being one possible exception.

(2) ALMOST-ANY FIT CONSTRAINT. If BIN_j is the *unique* bin with lowest nonzero level, it cannot be chosen unless b will not fit in any bin to the left of BIN_j .

Let AF be the set of all packing rules that obey the first constraint, and AAF the more restricted set of all rules obeying both constraints. The members of AF and AAF , when thought of as on-line algorithms, will be the ANY FIT and ALMOST ANY FIT algorithms, respectively.

In defining AF and AAF we do not restrict ourselves to packing rules which, like FF and BF , are completely determined. In certain applications there might be extraneous considerations arising at runtime that might lead us to want to have a number of alternatives each time a piece is to be packed. For instance, in a dynamic paging environment we might not know in advance which pages (bins) are going to be in low speed memory, and hence can only be fetched at high cost. We extend our definitions to include packing rules and algorithms which allow us such choices by letting

$S(L)$ = the maximum number of bins used by any packing of L by S .

The following lemmas tell us some useful properties of AF - and AAF -packings.

LEMMA 2.1. Suppose PS is an S -packing for $S \in AF$, $b = \text{piece}_{PS}(j, 1)$, $1 \leq j' < j$, and $h \geq 1$. If

$$\text{size}(b) + \sum_{i=1}^{h-1} \text{size}(\text{piece}_{PS}(j', i)) \leq 1,$$

then there is a piece $b' = \text{piece}_{PS}(j', h)$, with $\text{rank}(b') < \text{rank}(b)$.

Proof. Immediate from the definition of AF , since b would have fit as the h 'th piece in BIN_j unless there already were such a piece in that bin when b was assigned. ■

LEMMA 2.2. If PS and b are as above, then for all j' , $1 \leq j' \leq j$,

$$\text{rank}(\text{piece}_{PS}(j', 1)) < \text{rank}(b).$$

Proof. Corollary of Lemma 2.1 with $h = 1$. ■

LEMMA 2.3. Suppose for $m \geq 2$ that L is a list with $\text{Range}(\text{size}_L) \subset (0, 1/m]$,

$S \in AF$, and PS is an S -packing of L . If $b = \text{piece}_{PS}(j, 1)$ for $j \leq \#PS$, then when b was assigned we had for all j' , $1 \leq j' < j$,

- (a) $\text{level}_P(j') > (m - 1)/m$ and
- (b) $\text{height}_P(j') \geq m$.

Proof. (a) follows from the facts that S obeys the AF constraint and that no piece exceeds $1/m$. (b) follows from (a) and the fact that $m - 1$ times $1/m$ cannot exceed $(m - 1)/m$. ■

LEMMA 2.4. Suppose L , S , and PS are as above and in addition $S \in AAF$, $1 \leq j < \#PS$, and $d = \max\{\text{gap}_{PS}(j') \mid 1 \leq j' < j\}$. Then BIN_j contains m pieces larger than d in PS .

Proof. Since $j + 1 \leq \#PS$, Lemma 2.3 tells us that BIN_j must contain at least m pieces. Let $b = \text{piece}_{PS}(j, h)$, $1 \leq h \leq m$.

If $h = 1$, $\text{size}(b) > d$ by Lemma 2.1. If $2 \leq h \leq m$, then at the time b was assigned, although BIN_j was not empty, it contained fewer than m pieces. Thus by Lemma 2.3, $\text{piece}_{PS}(j + 1, 1)$ could not yet have been assigned and so BIN_j must have been the right-most nonempty bin at the time. Since $\text{piece}_{PS}(j, 1)$ had already been assigned, all previous bins must already have had $\text{level}_P(j') > (m - 1)/m$, again by Lemma 2.3. Since BIN_j can have had $\text{level}_P(j)$ at most $(m - 1)/m$, it was hence the unique bin with lowest nonzero level, and so b must have been larger than all preceding gaps (and hence d) by the AAF constraint. ■

The next lemma, due to Demers [4], will allow us to use Results (1.1) and (1.2) to derive lower bounds on $R[S, t]$ for $S \in AF$.

LEMMA 2.5. If $S \in AF$ and $0 < t \leq 1$, then

$$R[S, t] \geq R[FF, t].$$

Proof. For any list L there is a permutation L' such that $S(L') = FF(L)$ for all $S \in AF$. Merely start L' with the pieces in the first bin of the FF -packing of L , followed by the pieces in the second bin, etc. When S is applied to L' , it will be forced to recreate the FF -packing of L . Therefore $S(L') \geq FF(L)$, $R[S, t](k) \geq R[FF, t](k)$, and the result follows. ■

Lemma 2.5 implies that in a sense FIRST FIT is the "best" of the AF algorithms. However, the following two theorems indicate that the fact that FF obeys Constraints (1) and (2) is more important than its particular method for choosing an assignment.

THEOREM 2.6. If $S \in AAF$, and $m = \lfloor 1/t \rfloor \geq 2$, then $R[S, t] = 1 + 1/m$.

Proof. Lemma 2.5 and Result (1.2) for *FF* imply that $R[S, t] \geq 1 + 1/m$. To prove that the inequality also goes the other way, consider an *S*-packing *PS* of a list *L*, none of whose pieces exceed $t \leq 1/m$.

CLAIM. For at most one BIN_j other than the last bin in *PS* is $\text{level}_{PS}(j) \leq m/(m+1)$.

Let $\text{BIN}_{j'}$ be the leftmost bin such that $\text{level}_{PS}(j') \leq m/(m+1)$, and hence $\text{gap}_{PS}(j') \geq 1/(m+1)$. Then by Lemma 2.4, for every $j, j' < j < \#PS$, BIN_j contains m pieces all with size $> \text{gap}_{PS}(j') \geq 1/(m+1)$, and hence $\text{level}_{PS}(j) > m/(m+1)$.

Thus there can be at most one bin other than the last whose level does not exceed $m/(m+1)$, and so we must have that the sum of all the pieces in *L*,

$$W(L) > [m/(m+1)] \cdot [S(L) - 2].$$

But clearly we must also have $L^* \geq W(L)$, and so

$$S(L) < \frac{(m+1)}{m} L^* + 2.$$

Thus $R[S, t](k) < (m+1)/m + 2/k$, and the result follows. ■

THEOREM 2.7. If $S \in AAF$ and $t > 1/2$, then $R[S, t] = 17/10$.

The lower bound follows from Lemma 2.5 and Result (1.1). The upper bound is obtained by a slight, though nontrivial, modification to Ullman's original proof for *FF* [5, 11]. Since the revised proof repeats almost all of his arguments, it will be omitted here, although it may be found in [7, 8]. The modification is based on the observation that, due to the *AAF* constraint, no piece placed in BIN_j before $\text{level}_p(j) > 1/2$ can have fit in any of the bins to the left of BIN_j .

To show that the above theorems do not extend to the larger class of *AF* algorithms, consider the following packing rule which is a member of *AF* — *AAF*.

WORST FIT (WF). If *b* will fit in the bin with the lowest nonzero level, put it there. Otherwise, choose the left-most empty bin.

THEOREM 2.8. (1) $R[WF, t] \geq 2$, for $t > 1/2$, and

(2) $R[WF, t] \geq 1 + 1/(1/t - 1)$, for $t \leq 1/2$.

Proof. We give examples of arbitrarily long lists *L* for which $WF(L)/L^*$ approaches these bounds. For (1), consider lists of the form

$$L = 2k \text{ repetitions of } \langle 1/2, 1/(2k) \rangle.$$

The optimal and *WF*-packings are given in Fig. 1, yielding

$$R[WF, t](k) \geq 2k/(k + 1) = 2 - 2/(k + 1).$$

with (1) following in the limit.

For (2), choose any $\epsilon > 0$ and let n be large enough so that $(1 - 2t)/n \leq \min(\epsilon, t)$. Consider lists of the form $L = \langle 2N \text{ repetitions of } \langle t, \langle n + 1 \text{ reps of } (1 - 2t)/n \rangle \rangle$.

Figure 2 shows optimal and *WF*-packings of such lists. Note that each bin in the *WF*-packing has level $t + n[(1 - 2t)/n] + (1 - 2t)/n \leq 1 - t + \epsilon$, and so we must have $W(L) \leq WF(L) [1 - t + \epsilon]$. The optimal packing is as shown, since the number of pieces of size $(1 - 2t)/n$ is $2N(n + 1) > Nn$, which is the number of such pieces required to fill up the N bins containing two pieces of size t , each of which requires exactly n of the pieces. The first N bins all have level 1, and all the remaining bins

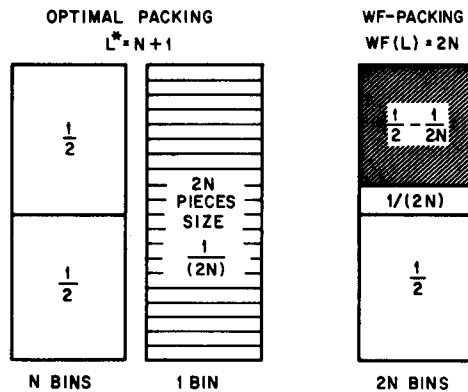


FIG. 1. Packings of list L in Theorem 2.8 (1).

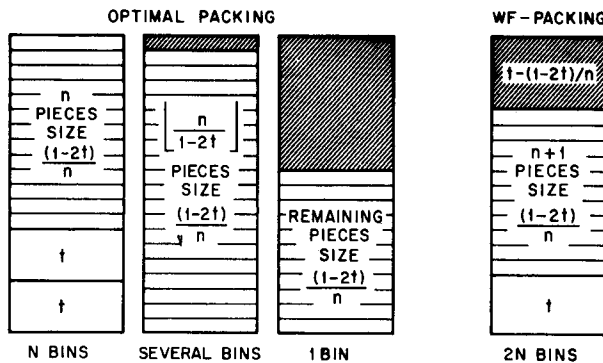


FIG. 2. Packings of List L in Theorem 2.8 (2).

except the last have as many pieces of size $(1 - 2t)/n$ as will fit and hence level exceeding $1 - \epsilon$. Thus $W(L) \geq (L^* - 1)(1 - \epsilon)$. Therefore

$$\begin{aligned} WF(L) &\geq (L^* - 1)[1 - \epsilon]/[1 - t + \epsilon] \\ &\geq (L^* - 1)[1/(1 - t) - 6\epsilon] \end{aligned}$$

(since $t \leq 1/2$), and so by Lemma 2.1, $R[WF, t] \geq 1 - t - 6\epsilon$. Since ϵ can be arbitrarily small, the lower bound follows. ■

Remark. The significance of Theorems 2.6 and 2.7 appears when we realize that by slightly modifying *WF* so that we try the bin with the *second* lowest level first, and only if *b* fails to fit there do we try the bin with the lowest level, we change *WF* into an *AAF* algorithm (ALMOST WORST FIT), and obtain a substantial improvement in worst case behavior.

WORST FIT is in a sense, however, the "worst" *AF* algorithm, for we have the following theorem.

THEOREM 2.9. *If $S \in AF$, then*

- (1) $17/10 \leq R[S, t] \leq 2$, for $t > 1/2$ and
- (2) $1 + 1/[1/t] \leq R[S, t] \leq 1 + 1/(1/t - 1)$, for $t \leq 1/2$.

Proof. The lower bounds follow from Lemma 2.5 and Results (1.1) and (1.2). For the upper bound in (1), note that in an *S*-packing of a list *L* there can be at most one bin with level less than $1/2$, and hence $L^* \geq W(L) \geq (1/2)(S(L) - 1)$ and so $R[S, t](k) \leq 2 + 1/k$, and the bound follows in the limit.

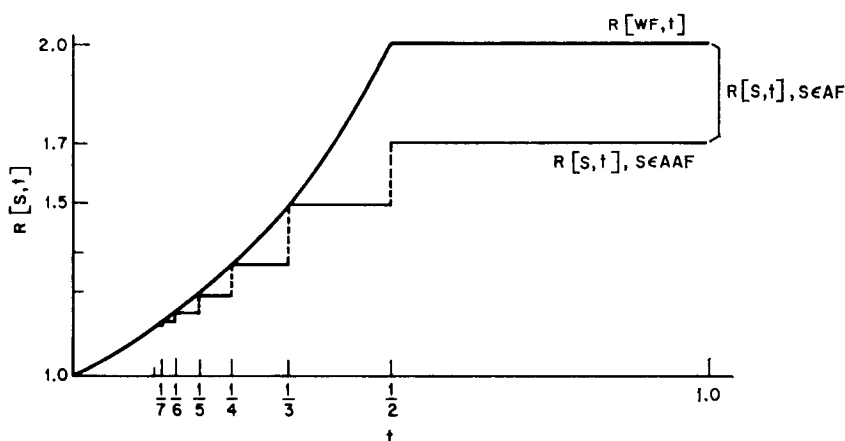


FIG. 3. $R[S, t]$ as a function of algorithm *S* and $t \in (0, 1]$.

For the upper bound in (2), note that by the *AF* constraint all bins except the last must have level exceeding $1 - t$. So in this case we must have

$$L^* \geq (1 - t)(S(L) - 1),$$

and hence

$$R[S, t](k) < \frac{1}{1 - t} + \frac{1}{k} = 1 + \frac{1}{1/t - 1} + \frac{1}{k},$$

and the bound follows in the limit. ■

The results of Theorems 2.6–2.9 are shown graphically in Fig. 3, where $R[S, t]$ is graphed as a function of $t \in (0, 1)$ for the various algorithms.

3. DECREASING ALGORITHMS

In the section, we consider algorithms consisting of an *AF* packing rule and the following preprocessing rule:

DECREASING RULE. L is in decreasing order, that is,

$$\text{size}(a) > \text{size}(b) \Rightarrow \text{rank}_L(a) < \text{rank}_L(b).$$

A list $L' = \langle b_1, b_2, \dots, b_n \rangle$ which has been reordered in accord with this rule will have $b_1 \geq b_2 \geq \dots \geq b_n$.

If S is a packing rule, the algorithm specified by S and the DECREASING RULE will be called S DECREASING or simply SD . We will be concerned with those SD where $S \in AF$, and we shall call the class of such algorithms *AFD*. The use of the DECREASING rule for preprocessing tends to lessen the difference between such algorithms. In [5] it was shown that if L is a list in decreasing order, all of whose pieces have size $\geq 1/5$, i.e., $\text{Range}(\text{size}_L) \subset [1/5, 1]$, then $FF(L) = BF(L)$. Perhaps even more surprising, in light of Theorem 2.8, is the fact that if $\text{Range}(\text{size}_L) \subset (1/4, 1]$, then for all $S, T \in AF$,

$$|SD(L) - TD(L)| \leq 1.$$

The current section will be devoted to proving this fact. We will then apply it in Section 4 to help us show that Result (1.3) extends to all *AFD* algorithms. First, we prove an important fact about *AF* packings of decreasing lists.

LEMMA 3.1. Suppose L is a list in decreasing order, $S \in AF$, and PS is an S -packing of L . If $\text{size}(\text{piece}_{PS}(j, 1)) \leq 1/m$, and $d = \max\{\text{gap}_{PS}(j') : 1 \leq j' < j\}$, then for every $j'', j \leq j'' < \#PS$,

(1) BIN_{j^*} contains at least m pieces of size $> d$, and

(2) for $1 \leq h \leq m$,

$$\text{size}(\text{piece}_{PS}(j'', h)) \geq \text{size}(\text{piece}_{PS}(j'' + 1, 1)).$$

Proof. Since BIN_{j^*} is not the last bin, we know there exists a piece $b = \text{piece}_{PS}(j'' + 1, 1)$. Let $a = \text{piece}_{PS}(j'', 1)$. By Lemmas 2.1 and 2.2, and the fact that L is in decreasing order,

$$d < \text{size}(b) \leq \text{size}(a) \leq 1/m.$$

By packing property (3) and the fact that L is in decreasing order, we know that all the pieces in BIN_{j^*} have size $\leq 1/m$. Thus if BIN_{j^*} contained $h < m$ pieces with rank less than that of piece b , they would fill the bottom h positions in BIN_{j^*} and we would have

$$\text{size}(b) + \sum_{i=1}^h \text{size}(\text{piece}_{PS}(j'', i)) \leq 1,$$

and yet $\text{piece}_{PS}(j'', h + 1)$, if it exists at all, would have rank $> \text{rank}(b)$, and so Lemma 2.1 would be violated. Thus the bottom m positions of BIN_{j^*} must be filled with pieces of smaller rank (and hence no smaller size) than piece b . Both (1) and (2) follow. ■

We now proceed directly to the main result of this section.

THEOREM 3.2. *If $S, T \in AF$ and L is a list in decreasing order with $\text{Range}(\text{size}_L) \subset (1/4, 1]$, then*

$$|S(L) - T(L)| \leq 1.$$

Proof. Let PS be an S -packing of L with $\#PS = S(L)$, and PT be a T -packing of L with $\#PT = T(L)$. Since L is in decreasing order, we can divide into segments $L = LA \cdot LB \cdot LC$, where

$$\text{Range}(\text{size}_{LA}) \subset (1/2, 1],$$

$$\text{Range}(\text{size}_{LB}) \subset (1/3, 1/2],$$

$$\text{Range}(\text{size}_{LC}) \subset (1/4, 1/3].$$

For $X \in \{A, B, C\}$, we shall refer to $\text{PIECES}(LX)$ as X -PIECES, and each $x \in X$ -PIECES as an X -piece. A bin in PS or PT whose bottom piece is an X -piece will be called an X -bin. By Lemma 3.1, in both PS and PT all the A -bins come to the left of all the B -bins, which in turn come to the left of all C -bins, and each B -bin (except possibly the right-most) contains two B -pieces, each C -bin (except possibly the right-most) contains three C -pieces. See Fig. 4.

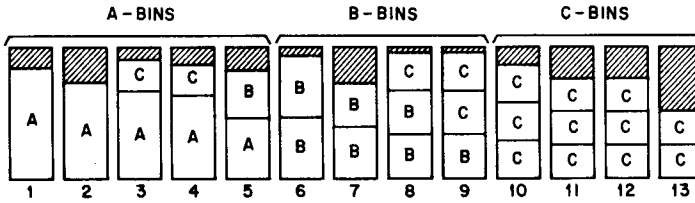


FIG. 4 Possible appearance of packing PS or PT in Theorem 3.2, with pieces labeled as to type.

CLAIM 3.2.1. *If BIN_j is an A -bin in either PS or PT , then*

$$\text{piece}_{PS}(j, 1) = \text{piece}_{PT}(j, 1).$$

Proof of Claim. The A -pieces head the list L , and since all have size exceeding $1/2$, they will be assigned consecutively, one per bin, each taking the bottom position in its bin, by any AF algorithm. ■

CLAIM 3.2.2. *If BIN_j is a B -bin in PS or PT , then*

$$\text{piece}_{PS}(j, 1) = \text{piece}_{PT}(j, 1).$$

Proof of Claim. Suppose BIN_j is a B -bin in PS and $b1 = \text{piece}_{PS}(j, 1)$. Let PSA be the segment of PS consisting of all A -bins, and PSB the segment of all B -bins to the left of BIN_j . Similarly, let PTA be the segment of PT consisting of all A -bins, and PTB the segment of all bins with index j' , $\#PTA < j' < j$. See Fig. 5.

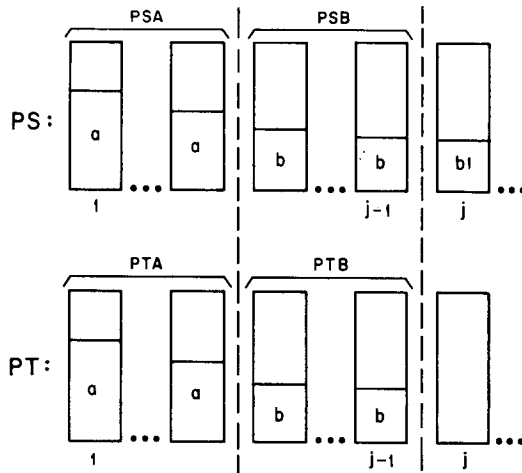


FIG. 5. Diagram for Claim 3.2.2.

In terms of these segments, we now define the following sets of pieces:

$$TB = \{b \in B\text{-PIECES} \cap \text{cont}(PTB) : \text{rank}(b) \leq \text{rank}(b1)\},$$

$$TA = \{b \in B\text{-PIECES} \cap \text{cont}(PTA) : \text{rank}(b) \leq \text{rank}(b1)\},$$

$$SB = \{b \in B\text{-PIECES} \cap \text{cont}(PSB) : \text{rank}(b) \leq \text{rank}(b1)\},$$

$$SA = \{b \in B\text{-PIECES} \cap \text{cont}(PSA) : \text{rank}(b) \leq \text{rank}(b1)\}.$$

First, note that $|SA| + |SB| + |A\text{-PIECES}| = \text{rank}(b1) - 1$, since all pieces with $\text{rank} < \text{rank}(b1)$ are in bins to the left of BIN_j , by Lemma 2.2 and packing property 3.

Next, since by Lemma 3.1 each bin in PSB must contain two B -pieces with $\text{rank} < \text{rank}(b1)$, and no bin can contain more than two B -pieces due to size constraints, we have $|SB| \geq |TB|$.

Finally, for each $b \in TA$, let $f(b)$ be the index of the A -bin containing b in PT . f is 1-1 since no A -bin can contain more than one additional piece, again by size constraints. We have, for all $b \in TA$,

$$\text{size}(b) + \text{size}(\text{piece}_{PT}(f(b), 1)) \leq 1.$$

By Claim 3.2.1 and the fact that $\text{rank}(b1) \geq \text{rank}(b) \Rightarrow \text{size}(b1) \leq \text{size}(b)$, we also have that for each $b \in TA$,

$$\text{size}(b1) + \text{size}(\text{piece}_{PS}(f(b), 1)) \leq 1.$$

Thus by Lemma 2.1 and the fact that f is 1-1, for each $b \in TA$ there must be a distinct $b' = \text{piece}_{PS}(f(b), 2)$ in SA , and so $|SA| \geq |TA|$.

Combining these inequalities, we get

$$|TA| + |TB| + |A\text{-PIECES}| \leq \text{rank}(b1) - 1.$$

But if BIN_j were empty in PT , or contained a bottom piece $b2$ with $\text{rank}(b2) > \text{rank}(b1)$ we would have to have

$$|TA| + |TB| + |A\text{-PIECES}| = \text{rank}(b1),$$

a contradiction. Thus there must be a $b2 = \text{piece}_{PT}(j, 1)$ with $\text{rank}(b2) \leq \text{rank}(b1)$.

By symmetry we can also conclude that $\text{rank}(b1) \leq \text{rank}(b2)$, so $b1 = b2$ and the claim is proved. ■

CLAIM 3.2.3. $\#PS \leq \#PT + 1$.

Proof of Claim. Let $j = \#PS$. If BIN_j is an A - or B -bin, or the left-most C -bin of PS , we are done by Claims 3.2.1 and 3.2.2. So we may assume that BIN_{j-1} is not an A - or B -bin in PT , and divide packings PS and PT into segments as follows:

$$PSA = \{A\text{-bins of } PS\},$$

$$PSB = \{B\text{-bins of } PS\},$$

$$PSC = \{C\text{-bins of } PS \text{ to the left of } \text{BIN}_j\},$$

$$PTA = \{A\text{-bins of } PT\},$$

$$PTB = \{B\text{-bins of } PT\},$$

$$PTC = \{\text{bins of } PT \text{ with index } j', \#PTA + \#PTB < j' < j-1\}.$$

See Fig. 6. Now let $c1 = \text{piece}_{PS}(j, 1)$ and

$$\mathcal{B} = \{b \in (B\text{-PIECES} \cup C\text{-PIECES}) : \text{rank}(b) \leq \text{rank}(c1)\},$$

and define the following sets of pieces:

$$TC = \mathcal{B} \cap \text{cont}(PTC),$$

$$TBC = \{b \in \mathcal{B} : b = \text{piece}_{PT}(j', 3) \text{ for some } \text{BIN}_{j'} \in PTB\},$$

$$TBB = \{b \in \mathcal{B} : b = \text{piece}_{PT}(j', h) \text{ for } h \in \{1, 2\} \text{ and for some } \text{BIN}_{j'} \in PTB\},$$

$$TA = \mathcal{B} \cap \text{cont}(PTA),$$

$$SC = \mathcal{B} \cap \text{cont}(PSC),$$

$$SBC = \{b \in \mathcal{B} : b = \text{piece}_{PS}(j', 3) \text{ for some } \text{BIN}_{j'} \in PSB\},$$

$$SBB = \{b \in \mathcal{B} : b = \text{piece}_{PS}(j', h) \text{ for } h \in \{1, 2\} \text{ and for some } \text{BIN}_{j'} \in PSB\},$$

$$SA = \mathcal{B} \cap \text{cont}(PSA).$$

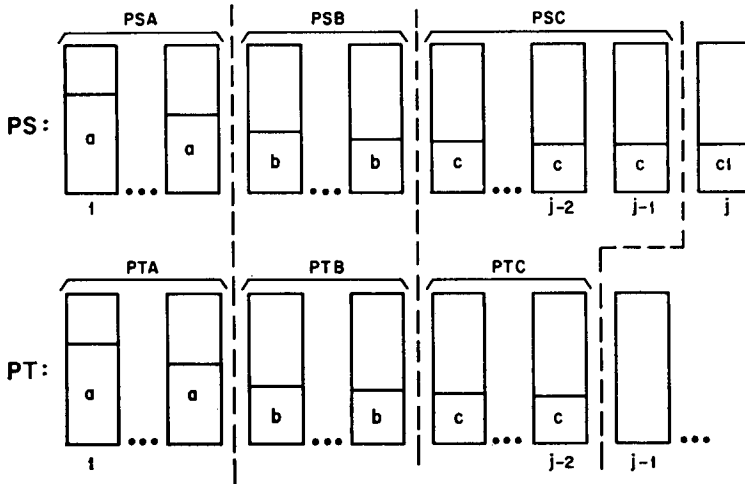


FIG. 6. Diagram for Claim 3.2.3.

We now can proceed with an analog of the argument for the previous claim. We first note that since, due to size constraints, no bin can contain more than three pieces from L , and no A -bin can contain more than two,

$$|A\text{-PIECES}| + |SA| + |SBB| + |SBC| + |SC| = \text{rank}(c1) - 1.$$

Next, we have that $|TBB| \leq |SBB| = 2(\#PSB)$, and $|TC| + 3 \leq |SC| = 3(\#PSC)$, since in the latter case, PSC contains one more bin than does PTC . Also, by the same argument we used in the preceding claim, $|TA| \leq |SA|$. All that remains to be considered are SBC and TBC .

For each $b \in TBC$, let $f(b)$ be the index of the B -bin containing b in PT . If $f(b) < \#PTA + \#PTB$, then $\text{BIN}_{f(b)+1}$ is in PSB . Let

$$\begin{aligned} b1 &= \text{piece}_{PT}(f(b), 1), \\ b2 &= \text{piece}_{PT}(f(b), 2), \\ b3 &= \text{piece}_{PS}(f(b) + 1, 1) = \text{piece}_{PT}(f(b) + 1, 1), \\ b4 &= \text{piece}_{PS}(f(b) + 1, 2). \end{aligned}$$

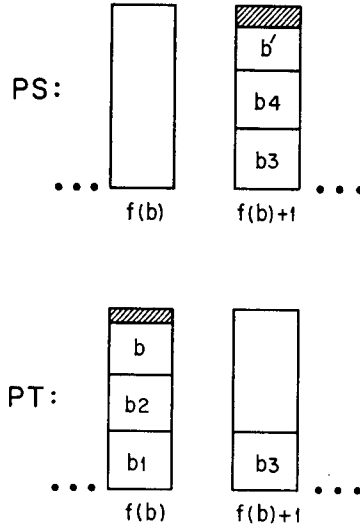


FIG. 7. Diagram for Claim 3.2.3.

See Fig. 7. By Lemma 3.1 and the fact that L is in decreasing order, $\text{size}(b1) \geq \text{size}(b2) \geq \text{size}(b3)$, and by packing property (3), $\text{size}(b4) \leq \text{size}(b3)$. Therefore,

$$\text{size}(b3) + \text{size}(b4) + \text{size}(c1) \leq \text{size}(b1) + \text{size}(b2) + \text{size}(b) \leq 1,$$

and by Lemma 2.1 there must be a $b' = \text{piece}_{PS}(f(b) + 1, 3)$ with $\text{rank}(b') < \text{rank}(c1)$. Since this is true for all $b \in TBC$ except the one possible b with $f(b) = \#PTA + \#PTB$, and since f is clearly 1-1, we thus have $|SBC| \geq |TBC| - 1$, and so can conclude that

$$|TA| + |TBB| + |TBC| + |TC| + |A\text{-PIECES}| \leq \text{rank}(c1) - 3.$$

But if BIN_{j-1} were empty in PT we would have to have

$$|TA| + |TBB| + |TBC| + |TC| + |A\text{-PIECES}| \geq \text{rank}(c1),$$

a contradiction. Thus there must be a $c2 = \text{piece}_{PT}(j - 1, 1)$ and so $\#PT \geq \#PS - 1$, proving the claim. ■

Since the proof of Claim 3.2.3 is equally valid if we interchange S and T , we also have $\#PS \geq \#PT - 1$, and the theorem follows. ■

Figure 8 gives an example of a decreasing list L with $\text{Range}(\text{size}_L) \subset (1/4, 1]$ for which $WF(L) = FF(L) + 1$, so Theorem 3.2 gives the best bounds possible. However, we *can* prove the following slightly more precise result.

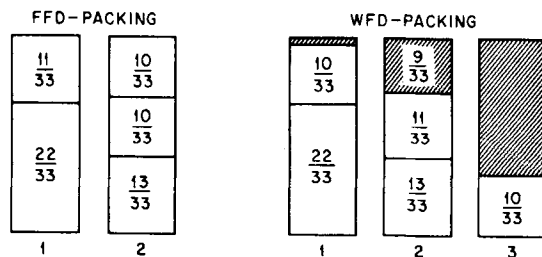


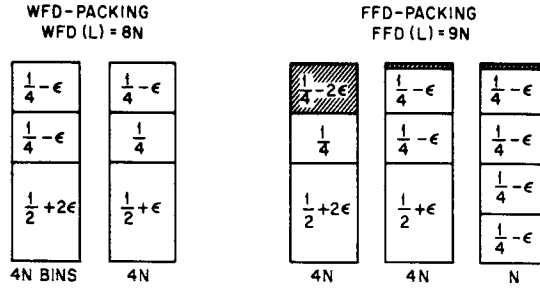
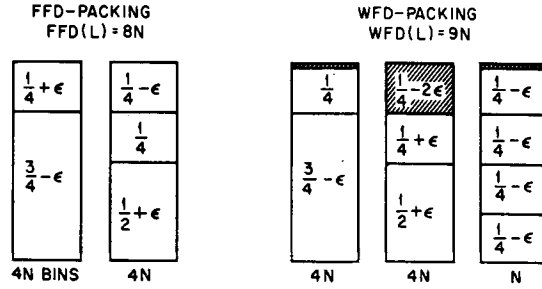
FIG. 8. Packings of list L for which $\text{Range}(\text{size}_L) \subset (\frac{1}{4}, 1]$ and $WF(L) = FF(L) + 1$.

THEOREM 3.3. *If $S \in AAF$, $T \in AF$, and L is a list in decreasing order with $\text{Range}(\text{size}_L) \subset (\frac{1}{4}, 1]$, then $S(L) \leq T(L) \leq S(L) + 1$.*

Remark. The near-equality concluded in Theorems 3.2 and 3.3 need not hold if the pieces in L are not restricted to the required range. However, there is still no evidence of the superiority or inferiority of any particular AFD algorithm. For instance, recall that in a sense WORST FIT was the worst AF algorithm, considerably worse than FF . However, WORST FIT DECREASING seems to be on equal footing with FFD . For every list we have tried we have found that

$$8/9 \leq FFD(L)/WFD(L) \leq 9/8,$$

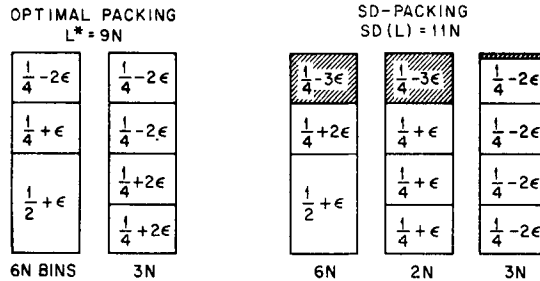
and both the bounds can be attained, as seen in Figs. 9 and 10.

FIG. 9. Packings of list L for which $FFD(L)/WFD(L) = 9/8$.FIG. 10. Packings of list L for which $FFD(L)/WFD(L) = 8/9$.

4. WORST CASE BEHAVIOR OF AFD ALGORITHMS

The results in Section 3 suggest that all *AFD* algorithms might have the same asymptotic worst case behavior. The goal of this section will be to show that this behavior at least lies within very narrow bounds, that in fact for all $S \in AF$,

$$11/9 \leq R[SD] \leq 5/4.$$

FIG. 11. Packings of list L for which $SD/L^* = 11/9$, $S \in AF$.

The lower bound follows from examples which originally appeared in [5] for *FFD* but apply to all *AFD* algorithms, and are given in Fig. 11. Using ideas derived in part from the following upper bound proof, we have in fact been able to show that if $S = FF$ or BF , $R[SD] = 11/9$ [7, 8]. The proof method does not extend to arbitrary $S \in AF$, however, and although we conjecture that the $11/9$ value does hold for all such algorithms, the result proved in this section is the best we have been able to prove.

We start with two lemmas about *FF* which will be of technical importance. The first version is a stronger version of Lemma 1.1, tailored to *FF* and decreasing lists.

LEMMA 4.1. *Suppose L is a list in decreasing order, PF is the *FF*-packing of L , $b \in \text{cont}_{PF}(j)$, $1 \leq j' < j$, and $h \geq 1$. If*

$$\text{size}(b) + \sum_{i=1}^{h-1} \text{size}(\text{piece}_{PF}(j', i)) \leq 1,$$

then there is a $b' = \text{piece}_{PF}(j', h)$ with $\text{size}(b') \geq \text{size}(b)$.

Proof. When b was assigned by *FF*, it went to the right of BIN_j . Since it would have fit as the h -th piece had the bin only contained $h - 1$ pieces at the time, the *FF* rule would have been violated unless position (j', h) were already filled by some piece $b' = \text{piece}_{PF}(j', h)$. But then we must have $\text{rank}(b') < \text{rank}(b)$, and hence $\text{size}(b') \geq \text{size}(b)$. ■

LEMMA 4.2. *Suppose L is a list in decreasing order with $\text{Range}(\text{size}_L) \subset (1/3, 1]$. Then $FF(L) = L^*$.*

Proof. The argument is quite similar to that for claim 3.2.2 in the proof of Theorem 3.2, and will be omitted. ■

We are now prepared to prove the major result of this section, that if $S \in AF$, $R[SD] \leq 5/4$. The following two lemmas reduce the problem to one about *FF*.

LEMMA 4.3. *Suppose $L = L1 \cdot L2 \cdot L3$ is a list in decreasing order with $\text{Range}(\text{size}_{L1}) \subset (1/4, 1]$, $\text{Range}(\text{size}_{L2}) \subset (1/5, 1/4]$, and $\text{Range}(\text{size}_{L3}) \subset (0, 1/5]$, and that $S \in AF$. If $FF(L1) \leq (L1 \cdot L2)^*$, then $S(L) \leq (5/4)L^* + 2$.*

Proof. Let PS be an S -packing of L using $S(L)$ bins, and divide it into four segments as shown in Fig. 12. $PS1$ is the first L^* bins, $PS2$ is BIN_{L^*+1} , $PS3$ is all bins to the right of $PS2$ with the exception of the last nonempty bin $\text{BIN}_{S(L)}$, which by itself makes up $PS4$. We may assume that $PS3$ and $PS4$ are both nonvacuous, else the claim would hold immediately. Thus $\#PS3 = S(L) - L^* - 2$.

For convenience, let us write $L12$ for $L1 \cdot L2$, and $L23$ for $L2 \cdot L3$.

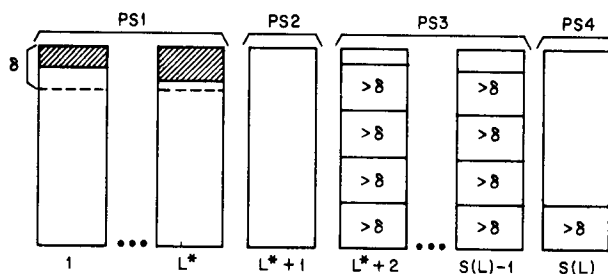


FIG. 12. Diagram for Lemma 4.3.

Now by Theorem 3.2 and the hypothesis, $S(L1) \leq FF(L1) + 1 \leq L12^* + 1 \leq L^* + 1$, so all the pieces of $L1$ are in bins of $PS1$ or $PS2$. Thus all the pieces in $\text{cont}(PS3)$ come from $L23$ and have size $\leq 1/4$.

Let $\delta = \max\{\text{gap}_{PS}(j) : \text{BIN}_j \text{ in } PS1\}$. Then $w(PS1) \geq (1 - \delta)L^*$. But by Lemma 3.1, every bin in $PS3$ contains at least four pieces of size exceeding δ , so $w(PS3) > 4\delta[S(L) - L^* - 2]$. Thus,

$$L^* \geq W(L) > w(PS1) + w(PS3) > (1 - \delta)L^* + 4\delta[S(L) - L^* - 2],$$

and so $S(L) < (5/4)L^* + 2$. ■

LEMMA 4.4. Suppose L and S are as above. If $FF(L1) + FF(L2) \leq (5/4)L^* + 2$, then $S(L) \leq (5/4)L^* + 3$.

Proof. Again let PS be an S -packing of L using $S(L)$ bins. This time divide it into three segments as shown in Fig. 13. $PS1$ is made up of the bins containing pieces from $L1$, $PS2$ those bins to the right of $PS1$ containing pieces from $L2$, and $PS3$ all remaining nonempty bins to the right of $PS2$.

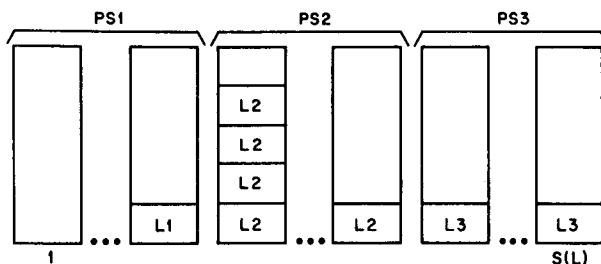


FIG. 13. Diagram for Lemma 4.4.

Now by Theorem 3.2 and the fact that all the pieces of $L1$ are contained in the bins of $PS1$,

$$\#PS1 \leq S(L1) \leq FF(L1) + 1.$$

Furthermore, since each bin in $PS2$ contains a piece from $L2$ and none contains a piece from $L1$, by Lemma 2.1 the bottom pieces in the bins of $PS2$ are all in $L2$. Thus by Lemma 3.1 each bin of $PS2$ except the last must contain at least four pieces of $L2$, and since *no* packing could contain more than four in a bin,

$$\#PS2 \leq \lceil |\text{PIECES}(L2)|/4 \rceil \leq FF(L2).$$

Thus, by hypothesis,

$$\#PS1 + \#PS2 \leq FF(L1) + 1 + FF(L2) \leq (5/4)L12^* + 3.$$

If $PS3$ is vacuous, $S(L) = \#PS1 + \#PS2$, and we are done. If not, then $\text{size}(\text{piece}_{PS}(S(L), 1)) \leq 1/5$, and so by Lemma 2.1, each of the first $S(L) - 1$ bins has level exceeding $4/5$. Thus $L^* \geq W(L) \geq (4/5)[(S(L) - 1)]$, and so $S(L) \leq (5/4)L^* + 1$. ■

In light of Lemma 4.3 and 4.4, the desired upper bound will follow from the next theorem.

THEOREM 4.5. *Suppose $L = L1 \cdot L2$ is a list in decreasing order with $\text{Range}(\text{size}_{L1}) \subset (1/4, 1]$ and $\text{Range}(\text{size}_{L2}) \subset (1/5, 1/4]$. Then*

$$FF(L1) > L^* \Rightarrow FF(L1) + FF(L2) \leq (5/4)L^* + 2.$$

Proof. The basic strategy of this proof is to show that if $FF(L1) > L^*$, then any optimal packing of L is too crowded with pieces from $L1$ for there to be very many pieces from $L2$ around. So assume $FF(L1) > L^*$, and let $L1 = LA \cdot LB \cdot LC$, where LA , LB , and LC are defined as in the proof of Theorem 3.2, and X -pieces, X -bins, etc., are also similarly defined for $X \in \{A, B, C\}$. In addition, we shall refer to the pieces of $L2 = LD$ as D -pieces. In summary we have

$$\begin{aligned} \text{Range}(\text{size}_{LA}) &\subset (1/2, 1], \\ \text{Range}(\text{size}_{LB}) &\subset (1/3, 1/2], \\ \text{Range}(\text{size}_{LC}) &\subset (1/4, 1/3], \\ \text{Range}(\text{size}_{LD}) &\subset (1/5, 1/4]. \end{aligned}$$

Since each A -bin can contain at most one A -piece a , we can identify it with that A -piece and call it the a -bin. This will allow us to compare the contents of particular a -bins between two different packings involving LA . A nonempty bin other than an A -bin will sometimes be referred to as a *non- A -bin*.

Now let P be the FF -packing of $L1$, and choose $QMAX$ and Q so that $QMAX < \#P$, and Q is a packing of $L1$ with $\max\{j: \text{level}_Q(j) > 0\} \leq QMAX$, and such that if in P BIN_j is the a -bin for $a \in A\text{-PIECES}$, then BIN_j is also the a -bin in Q . There must be such packings since by assumption $L1^* \leq L^* < FF(L)$, and any packing can have its bins rearranged so that its A -bins appear as required. For instance (and this trick will be applied later in the proof), we could get such a Q by taking an optimal packing of L and removing all the D -pieces.

Now divide P into segments PA , PB , and PX , and Q into segments QA and QB as follows (see Fig. 14):

$$\begin{aligned} PA &= A\text{-bins of } P = \{BIN_j: 1 \leq j \leq |A\text{-PIECES}|\}, \\ PB &= \{BIN_j: |A\text{-PIECES}| < j \leq QMAX\}, \\ PX &= \{BIN_j: \text{cont}_P(j) \neq \emptyset \text{ and } j > QMAX\}, \\ QA &= A\text{-bins of } Q = \{BIN_j: 1 \leq j \leq |A\text{-PIECES}|\}, \\ QB &= \{BIN_j: |A\text{-PIECES}| < j \leq QMAX\}. \end{aligned}$$

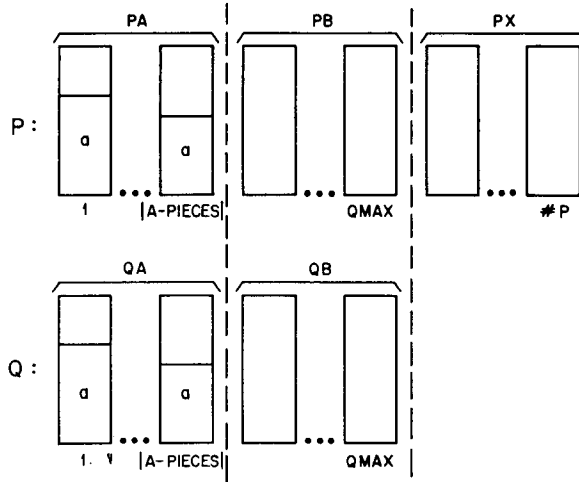


FIG. 14. Diagram for Theorem 4.5.

Note that some of the bins in QB may be empty, since we can have $QMAX > \#Q$; but since $\#P > QMAX$, segment PX is not vacuous. Moreover, PX cannot contain any B -pieces, since if it did we would have $FF(LA \cdot LB) > QMAX \geq L1^* \geq (LA \cdot LB)^*$, in violation of Lemma 4.2. Thus we have the following.

CLAIM 4.5.1. $\emptyset \neq \text{cont}(PX) \subseteq C\text{-PIECES}$.

Now let $\mathcal{B} = B\text{-PIECES}(L) \cup C\text{-PIECES}(L)$, and define the following subsets of $\mathcal{B}$:

$$\mathcal{Q} = \mathcal{B} - \text{cont}(QA),$$

$$\mathcal{P} = \mathcal{B} - \text{cont}(PA),$$

$$\mathcal{O} = \{x \in \mathcal{B} \cap \text{cont}(PA) : \text{the } A\text{-bin containing } x \text{ in } P \text{ has an } A\text{-piece with size} \\ < 3/5, \text{ and contains no element of } \mathcal{B} \text{ in } Q\}.$$

Letting $e = \text{piece}_P(Q\text{MAX} + 1, 1)$, we have $\text{size}(e) \leq 1/3 < 2/5$ by Claim 4.5.1. Thus e would have fit as the second piece in any A -bin whose A -piece had size $< 3/5$. Consequently, by Lemma 4.1, each such A -bin must contain a $\mathcal{B}$ -piece with $\text{rank} < \text{rank}(e)$. There can be no more than one by size constraints, so we have the following.

CLAIM 4.5.2. *If $a \in A\text{-PIECES}$, $\text{size}(A) < 3/5$, and the a -bin contains no $\mathcal{B}$ -pieces in Q , then it contains exactly one element of $\mathcal{O}$ in P , and that element has size $\geq \text{size}(e)$.*

We wish to show that the crowding of Q , due to the fact that it uses fewer bins than P and that the A -bins which contain elements of $\mathcal{O}$ in P contain no $\mathcal{B}$ -pieces in Q , is all concentrated in segment QB . To this end we shall define a 1-1 map f from $\mathcal{P} \cup \mathcal{O}$ to $\mathcal{Q}$.

The definition proceeds as follows: We say that a piece y *points to* a piece x if $x = \text{piece}_P(j, 2)$ and $y = \text{piece}_O(j, 2)$ for BIN_j an A -bin. A *chain* of distinct pieces is a sequence $\langle x_1, \dots, x_k \rangle$ such that for $1 \leq i < k$, x_i points to x_{i+1} . A *loop* is a chain $\langle x_1, \dots, x_k \rangle$ in which x_k points to x_1 . A *maximal chain* is a chain which is not a loop and not a proper subsequence of any other chain. If $\langle x_1, \dots, x_k \rangle$ is a maximal chain, x_1 is its *head*, and x_k is its *tail*.

For each $x \in \mathcal{P} \cup \mathcal{O}$, define $f(x) = \text{tail of maximal chain headed by } x$.

CLAIM 4.5.3. *f is a 1-1-map from $\mathcal{P} \cup \mathcal{O}$ to $\mathcal{P}$ satisfying*

$$(A) \quad x \in \mathcal{P} \Rightarrow \text{size}(f(x)) \geq \text{size}(x),$$

$$(B) \quad x \in \mathcal{O} \Rightarrow \text{size}(f(x)) \geq \text{size}(e).$$

Proof of Claim. If x is not pointed to by any piece, then x must be the head of some maximal chain, even if the chain is only $\langle x \rangle$, which would occur if x itself did not point to anything. Now if $x \in \mathcal{P}$, x is not in an A -bin in P , so cannot be pointed to. Similarly, if $y \in \mathcal{O}$, $\text{piece}_O(j, 2)$ is not defined for BIN_j the A -bin containing y in P , and so y is not pointed to. Thus f is well defined. Since no piece can be pointed to by more than one other piece, f is clearly 1-1.

That $\text{Range}(f) \subseteq \mathcal{Q}$ and properties (A) and (B) hold follow from a simple induction. Let $\langle x_1, \dots, x_k \rangle$ be a maximal chain headed by $x_1 \in \mathcal{P} \cup \mathcal{O}$. Our induction hypothesis is

$$(H) \quad \text{If } x_1 \in \mathcal{P}, \text{size}(x_i) \geq \text{size}(x_1).$$

$$\text{If } x_1 \in \mathcal{O}, \text{size}(x_i) \geq \text{size}(e).$$

If $i = 1$, (H) holds trivially for $x_i \in \mathcal{P}$, and by Claim 4.5.2 if $x_1 \in \mathcal{Q}$. Suppose it holds for x_i and $x_i \notin \mathcal{Q}$. Then $x_1 = \text{piece}_o(j, 2)$ for BIN_j , the a -bin for some $a \in A\text{-PIECES}$, so

$$\begin{aligned} \text{size}(x_1) + \text{size}(a) &\leq \text{size}(x_i) + \text{size}(a) \leq 1, \\ [\text{size}(e) + \text{size}(a) &\leq \text{size}(x_i) + \text{size}(a) \leq 1]. \end{aligned}$$

Since $x_1[e]$ is not in $\text{cont}(PFA)$, by Lemma 4.1 there is a piece $b = \text{piece}_p(j, 2)$ with $\text{size}(b) \geq \text{size}(x_1)$ [$\text{size}(b) \geq \text{size}(e)$]. Hence x_i points to b . Since $\langle x_1, \dots, x_k \rangle$ is a maximal chain, x_k cannot point to anything, so $i < k$, $b = x_{i+1}$, and (H) holds for x_{i+1} .

Thus by induction $x_k \in \mathcal{Q}$, $f(x_1) = x_k$ obeys (A) [(B)] and, Claim 4.5.3 is proved. ■

From this point on we shall ignore any pieces in $\mathcal{Q}\text{-Range}(f)$. The bins of QB will be crowded enough without them. In order to further emphasize this crowdedness, we shall use f to show that QB must contain a number of "big" pieces. For the moment let us return our attention to segment PB of P . Let

$$\begin{aligned} r &= \text{MAX}\{j: \text{BIN}_j \text{ in } PB \text{ and } \text{height}_p(j) = 2\}, \\ \text{BOT} &= \{\text{piece}_p(j, h): |A\text{-PIECES}(L)| < j \leq r, h \leq 2\} - \text{piece}_p(r, 2), \\ \text{TOP} &= \{\text{piece}_p(j, 3): |A\text{-PIECES}(L)| < j \leq r\}. \end{aligned}$$

See Fig. 15. Now since the sums of the sizes of the bottom two pieces in the bins of PB must form a nonincreasing sequence by Lemma 3.6, each $b \in \text{TOP}$ would fit in $\text{gap}_p(r)$. Thus every piece in a bin to the right of BIN_r in P must have rank $< \text{rank}(b)$ for all $b \in \text{TOP}$, and so the pieces in TOP must all be comparatively small. Now let

$$\begin{aligned} \text{BIG} &= f(\text{BOT}), \\ \text{SMALL} &= \{z \in \text{Range}(f): \text{there exist } x \neq y \in \text{BIG} \text{ with} \\ &\quad \text{size}(x) + \text{size}(y) + \text{size}(z) \leq 1\}. \end{aligned}$$

These two names will take on significance in the light of the next two claims.

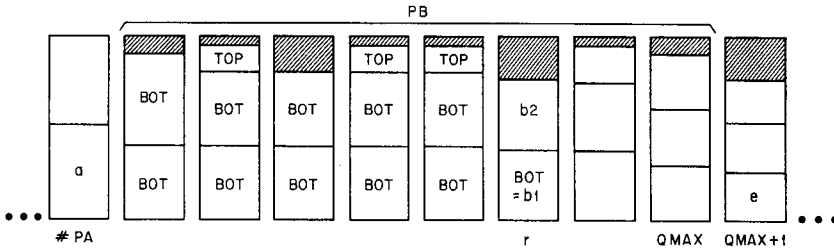


FIG. 15. Diagram for Theorem 4.5.

CLAIM 4.5.4. $BIG \subseteq B\text{-PIECES}(L)$.

Proof of Claim. If $\text{piece}_P(r, 1)$ were a C -piece, then by packing property (3) so would be $\text{piece}_P(r, 2)$ and we would have $\text{level}_P(r) \leq 2/3$. Thus piece e would have fit in BIN_r and could not have gone on to BIN_{OMAX+1} , contrary to the definition of e . Thus $\text{piece}_P(r, 1)$ is a B -piece, and hence by Lemma 3.1, so are all the other elements of BOT . The claim follows by Claim 4.5.3A. ■

CLAIM 4.5.5. $\text{SMALL} \subseteq f(\text{TOP})$.

Proof of Claim. Let $b1 = \text{piece}_P(r, 1)$, $b2 = \text{piece}_P(r, 2)$.

$$\text{size}(b1) \leq \min\{\text{size}(x) : x \in \text{BIG}\}$$

by Lemma 3.1 and Claim 4.5.3A, and $\text{size}(b2) \leq \text{size}(b1)$ by packing property (3). Therefore, if $x \in \text{SMALL}$, $\text{size}(x) + \text{size}(b1) + \text{size}(b2) \leq 1$, and so $\text{size}(x) \leq 1 - \text{size}(b1) - \text{size}(b2) = \text{gap}_P(r)$.

Consider $y = f^{-1}(x)$. There are four possibilities:

- (1) $y \in \text{cont}_P(j)$ for some $j > r$,
- (2) $y \in \mathcal{O}$,
- (3) $y \in \text{BOT} \cup \{b2\}$,
- (4) $y \in \text{TOP}$.

If $y \in \text{cont}_P(j)$ for $j > r$, then by the FF rule, $\text{size}(y) > \text{gap}_P(r) \geq \text{size}(x)$, contradicting Claim 4.5.3A, so $y \neq f^{-1}(x)$. [As a special case note that $\text{size}(e) > \text{gap}_P(r)$.]

If $y \in \mathcal{O}$, by Claim 4.5.3B, $\text{size}(x) = \text{size}(f(y)) \geq \text{size}(e) > \text{gap}_P(r)$, so $y \neq f^{-1}(x)$.

If $y \in \text{BOT} \cup \{b2\}$, by Lemma 3.1 $\text{size}(y) \geq \text{size}(e)$, so again by Claim 4.5.3A, $y \neq f^{-1}(x)$.

Thus $y = f^{-1}(x) \Rightarrow y \in \text{TOP}$, and the claim is proved. ■

We now do some explicit counting of the pieces in $\text{Range}(f)$. Let

$$p2 = |\{j: \text{BIN}_j \text{ in } PB \text{ and } \text{height}_P(j) = 2\}|.$$

$$p3 = |\{j: \text{BIN}_j \text{ in } PB \text{ and } \text{height}_P(j) = 3\}|.$$

CLAIM 4.5.6.

- (A) $p2 + p3 = \#QB$,
- (B) $|\text{Range}(f)| = 2(p2) + 3(p3) + |\text{cont}(PX)| + |\mathcal{O}|$.
- (C) $|\text{BIG}| = 2(p2 + |\text{TOP}|) - 1$,
- (D) $|\text{Range}(f) - \text{BIG}| = 3(p3) - 2|\text{TOP}| + 1 + |\text{cont}(PX)| + |\mathcal{O}|$.

Proof of Claim. $\text{Range}(\text{size}_{L1}) \subset (1/4, 1]$, so no bin can contain more than three pieces in P . $\text{BIN}_{Q_{\text{MAX}+1}}$ is not empty, so Lemma 3.1 says that every bin in PB must contain at least two pieces. Since $\#PB = \#QB$, (A) holds. (B) follows from Claim 4.5.3, (C) follows from the definitions of BIG and TOP, and (D) follows from (B) and (C). ■

We now return to the packing Q . Even though crowded, some of its bins may still have room left over for one or more D -pieces. Let D_{MAX} be the maximum number of pieces of size exceeding $1/5$ that could be placed in the gaps in the first Q_{MAX} bins of Q . To compute an upper bound on D_{MAX} , let us classify the bins of Q as to their contents. Let

$$A(0) = \{\text{BIN}_j : j \leq |A\text{-PIECES}|, \text{level}_Q(j) \geq 4/5\},$$

$$A(1) = \{\text{BIN}_j : j \leq |A\text{-PIECES}|, \text{level}_Q(j) \in [3/5, 4/5)\},$$

$$A(2) = \{\text{BIN}_j : j \leq |A\text{-PIECES}|, \text{level}_Q(j) < 3/5\},$$

$$B(i, k) = \{\text{BIN}_j \text{ in } QB :$$

$$|\text{cont}_Q(j) \cap \text{BIG}| = k,$$

$$|\text{cont}_Q(j) \cap \text{Range}(f)| = i\}.$$

In addition, let $a(i) = |A(i)|$, $b(i, k) = |B(i, k)|$. Then we have the following claim.

CLAIM 4.5.7.

$$(A) \quad \#QA = \sum_{i=0}^2 a(i),$$

$$(B) \quad \#QB = \sum_{k=0}^2 b(3, k) + \sum_{k=0}^2 b(2, k) + \sum_{k=0}^1 b(1, k) + b(0, 0).$$

$$(C) \quad D_{\text{MAX}} \leq [a(1) + b(3, 0) + b(2, 2)] + 2[a(2) + b(2, 1) + b(2, 0)] \\ + 3[b(1, 1) + b(1, 0)] + 4[b(0, 0)].$$

Proof of Claim. (A) is immediate. Since no bin can contain more than three pieces from $L1$, the only way $b(i, k)$ can be nonzero is if $0 \leq k \leq i \leq 3$. $b(3, 3) = 0$ since three BIG-pieces would have a total size exceeding $3(1/3)$ by Claim 4.5.5. Thus the right side of (B) contains all nonzero $b(i, k)$'s and so (B) holds.

(C) follows due to size constraints: In order for a bin to have room for k D -pieces, it must have a gap in Q of more than $k/5$. Let us consider the maximum possible gaps for each class of bins.

- $A(0)$: gap $\leq 1/5$, at most 0 D -pieces.
 $A(1)$: gap $\leq 2/5$, at most 1 D -piece.
 $A(2)$: gap $\leq 1/2$, at most 2 D -pieces.
 $B(3, 2)$: gap $\leq 1 - 2/3 - 1/4 = 1/12$, at most 0.
 $B(3, 1)$: gap $\leq 1 - 1/3 - 2/4 = 1/6$, at most 0.
 $B(3, 0)$: gap $\leq 1 - 3/4 = 1/4$, at most 1.
 $B(2, 2)$: gap $\leq 1 - 2/3 = 1/3$, at most 1,
 $B(2, 1)$: gap $\leq 1 - 1/3 - 1/4 = 5/12$, at most 2.
 $B(2, 0)$: gap $\leq 1 - 2/4 = 1/2$, at most 2.
 $B(1, 1)$: gap $\leq 1 - 1/3 = 2/3$, at most 3.
 $B(1, 0)$: gap $\leq 1 - 1/4 = 3/4$, at most 3.
 $B(0, 0)$: gap ≤ 1 , at most 4.

(C) is simply a summary of this case analysis, since as argued above the sets listed form a partition of the bins of QA and QB . ■

In order for the above bound on $DMAX$ to have much meaning, we must know something more about the values of the $a(i)$'s and $b(i, k)$'s. To do this we shall use the information about $\text{Range}(f)$ in Claim 4.5.6. For convenience, let us use the following shorthand notation:

$$B(i) = \bigcup_{k=0}^i B(i, k), \quad b(i) = |B(i)|.$$

We are especially interested in counting the three-piece bins containing a BIG-piece, $B(3, 1) \cup B(3, 2)$, for as we saw in the proof of Claim 4.5.7, these are the bins that do not have room for *any* D -pieces. We shall show that there are many of them. To this end, let

$$IN = \{x \in \text{BIG} \cap \text{cont}_O(j) : \text{BIN}_j \in B(3)\}.$$

By obtaining a value for $|IN|$, the number of BIG-pieces in bins of $B(3)$, we can then get a lower bound on the number of such bins containing BIG-pieces. An intuitive argument about the value of $|IN|$ goes as follows: Recall that PB is made up entirely of two- and three-piece bins, the number of BIG-pieces in three-piece bins in PB is simply $2|TOP|$, and each of the two-piece bins (except for BIN_*) contains two BIG-pieces. Thus for each piece that QB contains in excess of the number that PB contains, there must be an additional three-piece bin, and hence two additional BIG-pieces in IN . This makes for a total contribution of $2(|\text{cont}(PX)| + |\mathcal{A}|)$. Another possible reason for an increase in the number of three-piece bins in QB is that some bin which is not a three-piece bin contains fewer than two pieces. Such

deficient bins will thus cause two pieces to be added to IN for each piece they are deficient. And finally, if a bin which is not a three-piece bin does not contain two BIG-pieces, an additional piece is added to IN for each BIG-piece the bin is deficient. Thus each of the classes $B(i, k)$, $i \leq 2$, of bins not containing three pieces from $\text{Range}(f)$ will contribute $[2(2 - i) + (2 - k)] b(i, k)$ to $|\text{IN}|$. Let

$$\begin{aligned} \text{EXTRA} &= \sum_{i=0}^2 \sum_{k=0}^i [2(2 - i) + (2 - k)] b(i, k) \\ &= 6b(0, 0) + 4b(1, 0) + 3b(1, 1) + 2b(2, 0) + b(2, 1). \end{aligned}$$

The next two claims summarize these arguments and give more formal proofs.

CLAIM 4.5.8. $b(3) - p_3 = |\text{cont}(PX)| + |\mathcal{O}| + 2b(0, 0) + b(1)$.

Proof of Claim. By claims 4.5.6A and 4.5.7B,

$$p_2 + p_3 = b(0, 0) + b(1) + b(2) + b(3).$$

By Claim 4.5.6B and the definition of the $b(i, k)$'s,

$$\begin{aligned} b(1) + 2b(2) + 3b(3) &= |\text{range}(f)| \\ &= 2(p_2) + 3(p_3) + |\text{cont}(PX)| + |\mathcal{O}|. \end{aligned}$$

Claim 4.5.8 follows from combining these two equations. ■

CLAIM 4.5.9. $|\text{IN}| = 2[|\text{cont}(PX)| + |\mathcal{O}| + |\text{TOP}|] - 1 + \text{EXTRA}$.

Proof of Claim. Let

$$\begin{aligned} \text{IN}' &= \{x \in [\text{Range}(f) - \text{BIG}] \cap \text{cont}_O(j) : \text{BIN}_j \in B(3)\}, \\ \text{OUT}' &= \{x \in [\text{Range}(f) - \text{BIG}] \cap \text{cont}_O(j) : \text{BIN}_j \notin B(3)\}. \end{aligned}$$

As immediate consequences of the definitions we have

$$\begin{aligned} |\text{IN}| + |\text{IN}'| &= 3b(3), \\ |\text{IN}'| + |\text{OUT}'| &= |\text{Range}(f) - \text{BIG}|, \\ |\text{OUT}'| &= 2b(2, 0) + b(2, 1) + b(1, 0). \end{aligned}$$

Combining we get

$$|\text{IN}| = 3b(3) + 2b(2, 0) + b(2, 1) + b(1, 0) - |\text{Range}(f) - \text{BIG}|.$$

But by Claims 4.5.6B and 4.5.8,

$$\begin{aligned} |\text{Range}(f) - \text{BIG}| &= 3(p3) - 2|\text{TOP}| + 1 + |\text{cont}(PX)| + |\mathcal{O}|, \\ 3b(3) - 3(p3) &= 3[|\text{cont}(PX)| + |\mathcal{O}| + 2b(0, 0) + b(1)]. \end{aligned}$$

Claim 4.5.9 follows by substitution. ■

We now can give a lower bound on the number of three-piece bins containing BIG-pieces.

CLAIM 4.5.10.

$$b(3, 1) + b(3, 2) \geq 2[|\text{cont}(PX)| + |\mathcal{O}|] + |\text{TOP}| + \text{EXTRA} - 1.$$

Proof of Claim. By the definitions, we have $|\text{IN}| = b(3, 1) + 2b(3, 2)$. Therefore,

$$b(3, 1) + b(3, 2) = |\text{IN}| - b(3, 2).$$

But by Claim 4.5.5 and the definition of SMALL,

$$b(3, 2) \leq |\text{SMALL}| \leq |\text{TOP}|.$$

Using this to substitute for $b(3, 2)$ and Claim 4.5.9 to substitute for $|\text{IN}|$ yields the desired result. ■

We are now ready to get a more meaningful bound on DMAX.

CLAIM 4.5.11. $\text{DMAX} \leq \text{QMAX} - 2|\text{cont}(PX)| + 1$.

Proof of Claim. By Claim 4.5.7 we have

$$\begin{aligned} \text{QMAX} = \#QA + \#QB &= b(3, 2) + b(3, 1) + b(3, 0) + \sum_{i=0}^2 a(i) + \sum_{k=0}^2 b(2, k) \\ &\quad + \sum_{k=0}^1 b(1, k) + b(0, 0). \end{aligned}$$

Using Claim 4.5.10 to substitute for $b(3, 2) + b(3, 1)$ we get

$$\begin{aligned} \text{QMAX} &\geq 2|\text{cont}(PX)| - 1 + |\text{TOP}| + 2|\mathcal{O}| + b(3, 0) \\ &\quad + \sum_{i=0}^2 a(i) + 3b(2, 0) + 2b(2, 1) + b(2, 2) \\ &\quad + 4b(1, 1) + 5b(1, 0) + 7b(0, 0). \\ &\geq 2|\text{cont}(PX)| - 1 \\ &\quad + [a(1) + b(3, 0) + b(2, 2)] + 2[a(2) + b(2, 1) + b(2, 0)] \\ &\quad + 3[b(1, 1) + b(1, 0)] + 4[b(0, 0)], \end{aligned}$$

since $|\mathcal{A}| = a(2)$ by Claim 4.5.2. But by Claim 4.5.7C, this yields $Q_{\text{MAX}} \geq 2|\text{cont}(PX)| - 1 + D_{\text{MAX}}$, and Claim 4.5.11 follows. ■

The next claim will complete the proof of Theorem 4.5.

CLAIM 4.5.12. $FF(L1) + FF(L2) \leq (5/4)L^* + 2$.

Proof of Claim. Suppose Q_{MAX} and Q were obtained as follows: Let P^* be an optimal packing of $L = L1 \cdot L2$. Obtain Q from P^* by removing all the D -pieces (the pieces from $L2$). Set $Q_{\text{MAX}} = L^* < \#P$. By the definition of D_{MAX} and Claim 4.5.11, we then have

$$|\text{PIECES}(L2)| \leq D_{\text{MAX}} \leq L^* - 2|\text{cont}(PX)| + 1.$$

Now since by Claim 4.5.1 all pieces in $\text{cont}(PX)$ have size in $(1/4, 1/3]$, and by definition $\text{Range}(\text{size}_{L2}) \subset (1/5, 1/4]$, Lemma 3.1 tells us that

$$\begin{aligned} \#PX &= \left\lceil \frac{|\text{cont}(PX)|}{3} \right\rceil < \frac{|\text{cont}(PX)|}{3} + 1, \\ FF(L2) &= \left\lceil \frac{|\text{PIECES}(L2)|}{4} \right\rceil < \frac{L^* - 2|\text{cont}(PX)| + 1}{4} + 1. \end{aligned}$$

Thus

$$\begin{aligned} FF(L1) + FF(L2) &= \#PA + \#PB + \#PX + FF(L2) \\ &= L^* + \#PX + FF(L2) \\ &\leq (5/4)L^* + 2 - |\text{cont}(PX)|/6 \leq (5/4)L^* + 2, \end{aligned}$$

and both Claim 4.5.12 and Theorem 4.5 are proved. ■

The immediate application of Theorem 4.5 is to prove our claimed upper bound on $R[SD]$.

THEOREM 4.6. *If $S \in AF$, $t \in (0, 1]$, then*

$$R[SD, t] \leq 5/4.$$

Proof. Let L be an arbitrary list in decreasing order, and divide it into segments $L1 \cdot L2 \cdot L3$, where $\text{Range}(\text{size}_{L1}) \subset (1/4, 1]$, $\text{Range}(\text{size}_{L2}) \subset (1/5, 1/4]$, and $\text{Range}(\text{size}_{L3}) \subset (0, 1/5]$. If $FF(L1) \leq (L1 \cdot L2)^*$, then by Lemma 4.3, $S(L) \leq (5/4)L^* + 2$. If not, then by Theorem 4.5, $FF(L1) + FF(L2) \leq (5/4)(L1 \cdot L2)^* + 2 \leq (5/4)L^* + 2$, and so by Lemma 4.4, $S(L) \leq (5/4)L^* + 3$. Thus the latter inequality holds in any case, and the theorem follows. ■

Theorems 3.2 and 4.5 can be generalized to give us upper bounds on $R[SD, t]$ for all $t \in (0, 1]$.

THEOREM 4.7. *If $S, T \in AF$ and L is a decreasing list with*

$$\text{Range}(\text{size}_L) \subset (1/(m+3), 1/m],$$

then

$$|S(L) - T(L)| \leq 1.$$

THEOREM 4.8. *If $S \in AF$ and $m = \lfloor 1/t \rfloor > 2$, then*

$$\frac{m+3}{m+2} - \frac{2}{m(m+1)(m+2)} < R[S, t] < \frac{m+3}{m+2}.$$

The upper bound in Theorem 4.8 follows from Theorem 4.7 in a generalization of the proof of Theorem 4.6 [7]. The lower bound follows from the example lists given in Fig. 16.

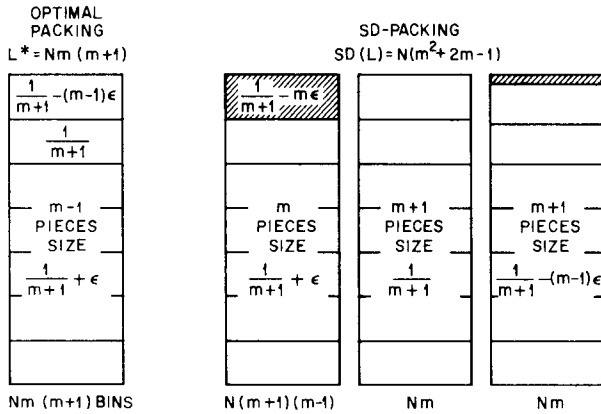


FIG. 16. Packings of list L for which $\text{Range}(\text{size}_L) \subset (0, 1/m]$ and $SD(L)/L^* = (m+3)/(m+2) - 2/(m(m+1)(m+2))$, $S \in AF$.

Remark. For $m = 2, 3$, and 5 the lower bounds given above are $7/6, 7/6$, and $23/20$. Our more sophisticated methods cited above have shown that in fact the actual values, at least for $S \in AAF$, are

$$\begin{aligned} R[SD, t] &= 71/60, & \text{for } 8/29 < t \leq 1/2, \\ &= 7/6, & \text{for } 1/4 < t \leq 8/29, \quad \text{and} \\ &= 23/20, & \text{for } 1/5 < t \leq 1/4. \end{aligned}$$

5. IMPLEMENTATION OF ANY FIT ALGORITHMS

In this section we begin by considering the problem of implementing the ANY FIT algorithms, with an eye toward determining how much time is required to generate the S -packing of a list L as a function of list length n .

We must talk about the “implementation” of our two-part algorithms because they have only been specified by rules, not explicit programs, and there may be more than one way to accomplish the desired effect of a given rule. That is, we have so far left open the problem of how to permute a given list so that it satisfies a preprocessing rule, or how to determine, given a set of bin levels and a piece size, which bin satisfies the requirements of a packing rule.

Although some of the algorithms we have studied have been assumed to operate on-line, we shall for the sake of the generality of our lower bound proof allow implementations which need not output any information about the packing until they have seen the entire list. They then must output a complete specification of the packing (or set of packings in the case of incompletely determined packing rules) that could result if the packing rule was applied as specified in our PART 2 program. We shall however, restrict our attention to *branching* implementations, ones which select the packing(s) by a sequence of yes–no tests, the result of each test determining either what test to make next or what packing(s) to output.

To be specific, given a list of length n , the S -packing (for S a two-part algorithm) is a sequence on n bins (some possibly empty). There are thus only a finite number of possible packings of lists of length n , each corresponding to a sequence of n disjoint sets whose overall union is $\{1, 2, \dots, n\}$, where the j th set consists of the ranks of the pieces in BIN_j of the packing. A branching implementation of a bin packing algorithm S for lists of length n will be a binary tree, whose internal nodes correspond to yes–no tests and whose leaves correspond to sets of possible packings of lists of length n . The tree must obey the property that any input list L of length n determines a path from the root to a leaf node by determining the answers to the tests at the internal nodes, and the leaf node at the end of the path corresponds to the set of packings generable by S applied to L . The *cost* for L of the implementation will be the number of internal nodes in the path.

This is the same type of theoretical framework within which the problem of *sorting* has been extensively analyzed, and all practical implementations of the algorithms would seem to fit into it, although of course the underlying tree structure will be implicit. Moreover, it is not unreasonable to expect that actual running times of such implementations would be proportional to the *cost*, as defined here.

We now briefly describe implementations of FF , BF , WF , and AWF , all of which do operate on-line, packing one piece at a time, and still have worst case cost bounded by a constant times $n \log n$.

FIRST FIT is implemented for list of length n by using a directory to the bins in

the form of a binary tree of depth $\lceil \log_2 n \rceil$ with n leaves corresponding to the n bins, in sequence from left to right. The leaf nodes are all labeled with the current gap for the corresponding bin, and each internal node is labeled with the larger of the labels of its two sons. It is easy to see how this directory can be used to find the first fit in no more than $\lceil \log_2 n \rceil$ comparisons (yes-no tests), and then be updated when the bin gap is updated, again using no more than $\lceil \log_2 n \rceil$ comparisons [7].

For *BF*, *WF*, and *AWF*, which unlike *FF* do not depend predominantly on the order of the bins in the original left to right sequence in making their assignments, we may use structures such as *AVL* trees [1] or trees of bounded balance [10] for our bin directories. These can keep the bins sorted by gap, and can be searched (to find the appropriate fit) and updated, both in time $O(\log n)$. The details are left to the reader.

Summarizing, we have the following theorem.

THEOREM 5.1. *For S any one of *FF*, *BF*, *WF*, and *AWF* there exists an implementation of S and a constant k such that if L is any list and its length is n , the cost for L of the implementation is at most $kn \cdot \log_2 n$.*

In the light of the following, we cannot expect to do much better.

THEOREM 5.2. *There exists a k such that for all $n > 4$, and any implementation of any *AF* algorithm for lists of length n , there exists a list L for which the cost is at least $kn \cdot \log_2 n$.*

Proof. Let $j = \lfloor n/2 \rfloor$. We shall assume n is even and $j = n/2$, but if n is odd, merely add a piece of size 1 to the lists we present. Consider the set of lists represented by

$$\langle x_1, \dots, x_j, 1/2 - d, 1/2 - 2d, \dots, 1/2 - jd \rangle,$$

where $\langle x_1, \dots, x_j \rangle$ is any permutation of $\langle 1/2 + d, 1/2 + 2d, \dots, 1/2 + jd \rangle$, and $0 < d < 1/2j$. Note that there are $j!$ such lists.

We claim that each is packed the same way by all *AF* algorithms, and no two yield the same packing. For consider the operation of S on one of the list. The first j pieces are all larger than $1/2$, so they go into BIN_1 thru BIN_j in the order they appear in the list. The next piece has size $1/2 - d$, and the only nonempty bin it will fit in will be the one containing the piece of size $1/2 + d$, which depends on the particular permutation, and the piece *must* go in that bin by the *AF* constraint. Then the next piece will only fit in the bin with the piece of size $1/2 + 2d$, and so on. Thus the bins which the last j pieces go into are completely determined by the permutation of the first j .

Therefore the tree representing the branching implementation must have at least $j! = \lfloor n/2 \rfloor!$ distinct leaf nodes, and hence the path to at least one of them must contain

at least $\lceil \log_2(\lfloor n/2 \rfloor!) \rceil$ internal nodes. The list whose S -packing corresponds to that leaf thus costs at least that much. By Stirling's formula there exists a k such that for all n , this exceeds $kn \cdot \log_2 n$. ■

The intuition behind the above result is that all AF algorithms implicitly require some form of sorting in their implementation. If we wish to develop linear time algorithms, we must somehow avoid this problem.

The following series of algorithms avoids the problem of the growing time for sorting by putting a fixed bound on the number of bins to be considered:

NEXT- k FIT (NkF): Assign b to BIN_j where
 $j = \text{MIN}\{j': \text{size}(b) \leq \text{gap}(j'), j' > \#P - k\}.$

NkF requires at most $kn - 1$ comparisons to generate a packing of a list of length n . In particular, $N1F$, the algorithm which starts a new bin unless the current piece will fit in the last nonempty bin of P , will use only $n - 1$ comparisons. $N1F$ has the same asymptotic worst case behavior as WORST FIT, proved in much the same way.

THEOREM 5.3. $R[N1F, t] = 2$, for $t \in (1/2, 1)$.

$$R[N1F, t] = \frac{1}{1-t}, \quad \text{for } t \in (0, 1/2).$$

On the other hand, for $k \geq 2$, NkF behaves more like an AAF algorithm. Since $k \geq 2$, no piece can be placed in position $(j+2, 1)$ unless it would not have fit in either BIN_{j+1} or BIN_j . Thus we have, similarly to Lemma 2.4.

LEMMA 5.4. Suppose L is a list with $\text{Range}(\text{size}_L) \subset (0, 1/m]$ for $m \geq 2$, and PN an NkF -packing of L for $k \geq 2$. If $1 < j < \#PN$ then positions $(j, 1)$ thru (j, m) are filled in PN by pieces with size $> \text{gap}_{PN}(j-1)$.

COROLLARY 5.4.1. If $\text{level}_{PN}(j) < m/(m+1)$, $1 \leq j \leq \#PN - 2$, then

$$\text{level}_{PN}(j) + \text{level}_{PN}(j+1) > 2m/(m+1).$$

Proof. Suppose $\text{level}_{PN}(j) = m/(m+1) - \epsilon$. Then $\text{gap}_{PN}(j) = 1/(m+1) + \epsilon$. By Lemma 5.4, since $j+1 < \#PN$, BIN_{j+1} must contain at least m pieces of size $> \text{gap}_{PN}(j)$, and hence have level $> m/(m+1) + m\epsilon$. ■

We can now show that for $k \geq 2$ and $t \in (0, 1/2)$, NkF indeed has the same worst case behavior as the AAF algorithms.

THEOREM 5.5. For $k \geq 2$ and $m = \lfloor 1/t \rfloor \geq 2$,

$$R[NkF, t] = 1 + 1/m.$$

Proof. The lower bound follows from that for FF , by the same reasoning we used in Lemma 2.5. For the upper bound, let L be a list, no piece of which has size exceeding $t \leq 1/m$, and let PN be a NkF -packing of L using $NkF(L)$ bins. A simple induction using Corollary 5.4.1 shows that

$$W(L) > \lfloor m/(m+1) \rfloor \cdot [NkF(L) - 2],$$

and the proof can be concluded as in Theorem 2.5. ■

The equality between $R[NkF, t]$ and $R[FF, t]$ concluded in the above theorem does not continue to hold for $t > 1/2$. In such situations we have $R[FF, t] = 17/10$.

THEOREM 5.6. For $k \geq 2$ and $t > 1/2$

$$R[NkF, t] \geq 17/10 + 3/(10k).$$

Proof. We build upon the construction for Theorem 1 [5] where the lower bound of $17/10$ for $R[FF]$ is developed. For each $N \equiv 0 \pmod{17}$ a list is given there consisting of $10N/17$ pieces of size about $1/6$, followed by $10N/17$ pieces of size about $1/3$, followed by $10N/17$ pieces of size $1/2 + \delta$ for $\delta > 0$. Under FF and NkF the first group of pieces fills up $2N/17$ bins, five pieces per bin, the next fills up the next $5N/17$ bins, two pieces per bin, and the final group fills up $10N/17$ bins, for a total of N bins. However, an optimal packing exists, using only $10N/17 + 1$ bins. We can improve on this example when the algorithm in question is NkF by interspersing additional pieces among the last $10N/17$ of the original list. The new list has the same initial $20N/17$ pieces and then proceeds as follows.

One new piece of size $1/2 - \delta/2$, one new piece of size $1/2 + \delta/1$, $k - 1$ old pieces of size $1/2 + \delta$, one new piece of size $1/2 - \delta/3$, one new piece of size $1/2 + \delta/2$, $k - 1$ old pieces, and so on; with the j th step placing one new piece of size $1/2 - \delta/(j+1)$, one new piece of size $1/2 + \delta/j$, and then $k - 1$ old pieces of size $1/2 + \delta$, until we have exhausted the $10N/17$ pieces from the original list.

We have thus added $2\lfloor (10N/17)/(k-1) \rfloor$ new pieces, each requiring its own bin in the NkF -packing, which consequently uses at least $N + (20/17)(N/(k-1))$ bins, whereas the optimal packing uses no more than $10N/17 + (10/17)(N/(k-1)) + 2$. The ratio of these two numbers approaches $17/10 + 3/(10k)$ as N approaches ∞ . ■

The remainder of this section will be devoted to another way of avoiding the problems with sorting, while still not suffering from a degradation of worst case behavior. The basic idea is to replace the direct comparisons of piece size versus bin gap or bin gap versus bin gap by comparisons against some fixed standards which serve to classify the pieces and gaps into groups all of whose members have similar sizes.

Let a *schedule of intervals* be a set $X = \{x_1, \dots, x_k\}$, where $x_1 = 0$, $x_k = 1$, and $x_i < x_{i+1}$, $1 \leq i < k$. X will be thought of as a partition of the unit interval $[0, 1]$ into the subintervals

$$[x_1, x_2), [x_2, x_3), \dots, [x_{k-1}, x_k), \{1\}.$$

$[x_i, x_{i+1})$ will be called *interval* X_i , with interval X_k being $\{1\}$.

Given a schedule of intervals X , we have the following packing rule.

GROUP-X FIT (GXF). To assign piece b , let $i' = \min\{i: x_i \geq \text{size}(b)\}$ and for some j , $1 \leq j \leq n$, $\text{gap}_p(j) \in X_{i'}$. Choose BIN_j such that $\text{gap}_p(j) \in X_{i'}$, subject to the constraint that if BIN_j is empty, $j = \min\{j': \text{level}_p(j') = 0\}$.

The final constraint is present only to insure that no piece is assigned to a bin which is to the right of an empty bin, a fact that does not in this case influence the number of bins used by the packing, but which *will* make our proofs more straightforward. Note that i' is chosen so that b will fit in *any* BIN_j such that $\text{gap}_p(j) \in X_{i'}$.

If Schedule X has k subintervals, *GXF* can be implemented using k stacks, $\text{STACK}_1, \dots, \text{STACK}_k$, one for each interval, and storing BIN_j (more precisely a pointer to a representation of BIN_j) on STACK_i if $\text{gap}_p(j) \in X_i$.

Initially, all bins are on STACK_k , ordered by index with BIN_1 on top. When b is to be assigned, we first find

$$i'' = \min\{i: x_i \geq \text{size}(b)\},$$

by a binary search using no more than $\lceil \log_2 k \rceil$ comparisons. We then search for

$$i' = \min\{i \geq i'': \text{STACK}_i \text{ is not empty}\},$$

pop the top bin on $\text{STACK}_{i'}$, and put b in the required position of that bin. Last, we find the proper stack on which to place the updated bin by a second binary search, again using no more than $\lceil \log_2 k \rceil$ comparisons.

Although the algorithm is clearly $O(n)$ for fixed k , we would like an implementation which is $O(n \log k)$, so that even for large k the constant of proportionality would not be unmanageable. Thus we cannot find i' given i'' by testing each stack in turn for nonemptiness, as this can require a number of tests linear in k .

Instead, we consider the stacks to be laid out in a line, and construct a binary tree over them with the stacks as leaf nodes, much as we did in our implementation of *FF*, only now the leaves correspond to stacks rather than individual bins, and each internal node of the tree need only store the information as to whether there is a nonempty stack beneath it in the tree. It is a simple matter to update the tree in $O(\log k)$ when a stack is changed, and also to use the tree as a directory and find i' given i'' using only $O(\log k)$ tests at internal nodes.

Thus the total cost of each piece assignment is $O(\log k)$ and the whole packing costs $O(n \log k)$.

Note that GXF operates by leaving a margin of safety. If we define $mesh(X)$ to be $\text{MAX}\{x_{i+1} - x_i : 1 \leq i < |X|\}$, then if $\text{gap}_P(j) > \text{size}(b) + \text{mesh}(X)$ we know that b can be assigned to BIN_j under GXF . On the other hand, if

$$\text{size}(b) > \text{gap}(j) - \text{mesh}(X),$$

it may be impossible to assign b to BIN_j even though it does fit. However, subject to the margin of error, GXF does try to put b in a bin so as to minimize the gap left. Thus as $\text{mesh}(X)$ approaches 0, GXF becomes more and more like BF , and we would expect its average case behavior to improve. (Note the similarity to the situation with NkF . As k increases, NkF becomes more and more like FF and might be expected to have better and better average case behavior.)

When we turn to the question of worst case behavior, we find that a surprisingly small schedule of intervals X is enough to make $R[GXF, t] = R[BF, t]$ for $t \leq 1/2$ (just as we had $R[NkF, t] = R[FF, t]$ for $t \leq 1/2$ as soon as $k \geq 2$). We shall reach this conclusion after presenting a series of lemmas. Our first lemma formalizes the most important fact about the way the algorithm operates.

LEMMA 5.7. *Suppose $y, z \in X$, $y < z$, and b is a piece in a list being packed by GXF , with $\text{size}(b) \leq y$. If when b was assigned there was a j' such that $\text{gap}_P(j') \in [y, z)$, then b was not assigned to any BIN_j with $\text{gap}_P(j) \geq z$.*

Proof. Suppose $y = x_{i1}$, $z = x_{i2}$. Then when b was assigned, $\text{BIN}_{j'}$ was on some $\text{STACK}_{i'}$, $i1 \leq i' < i2$, and BIN_j was on some STACK_i , $i \geq i2$. Since $\text{size}(b) \leq x_{i1}$, STACK_i could have been chosen only if all $\text{STACK}_{i''}$, $i1 \leq i'' < i$, were empty, and this is not the case. ■

Using Lemma 5.7, we can prove analogs of Lemmas 3.3 and 3.4 for AAF algorithms, under a weak restriction as to the nature of X .

LEMMA 5.8. *Suppose for $m \geq 2$ that $\{1/m, 1/(m+1)\} \subseteq X$, L is a list with $\text{Range}(\text{size}_L) \subset (0, 1/m]$, and PX is a GXF -packing of L . If $b = \text{piece}_{PX}(j, 1)$ for $j \leq \#PX$, then when b was assigned we had for all j' , $1 \leq j' < j$,*

$$(a) \quad \text{level}_P(j') > (m-1)/m \text{ and}$$

$$(b) \quad \text{height}_P(j') \geq m.$$

Proof of Lemma. For (a), suppose for some $j' < j$ $\text{level}_P(j') \leq (m-1)/m$ when b was assigned. $\text{BIN}_{j'}$ cannot have been empty at the time, since $j' < j$ and BIN_j was at the top of the empty bin stack. Thus $\text{gap}_P(j') \in [1/m, 1)$, $\text{gap}_P(j) \in \{1\}$, and $\text{size}(b) \leq 1/m$, so Lemma 5.7 would tell us that b did not go in $\text{BIN}_{j'}$, a contradiction. (b) follows from (a) since at least m pieces of size $\leq 1/m$ are required to yield a level $> (m-1)/m$. ■

LEMMA 5.9. Suppose L , X , and PX are as above and in addition $1 \leq j < \#PX$ and for some j' , $1 \leq j' < j$, $\text{gap}_{PX}(j') > 1/(m+1)$. Then BIN_j contains m pieces larger than $1/(m+1)$ in PX .

Proof of Lemma. Since $j+1 \leq \#PX$, Lemma 5.8 tells us that BIN_j must contain m pieces. Let $b = \text{piece}_{PX}(j, h)$, $1 \leq h \leq m$.

When b was assigned, BIN_j contained no more than $m-1$ pieces, and hence had $\text{gap}_P(j) > 1/m$. But again by Lemma 5.8, we know that $\text{gap}_P(j') < 1/m$, and hence by assumption $\text{gap}_P(j') \in [1/(m+1), 1/m)$. Thus if $\text{size}(b) \leq 1/(m+1)$, Lemma 5.7 would not allow b to go in BIN_j , and hence in fact $\text{size}(b) > 1/(m+1)$. ■

THEOREM 5.10. For $m = \lfloor 1/t \rfloor \geq 2$ and $\{1/(m+1), 1/m\} \subseteq X$

$$R[GXF, t] = 1 + 1/m.$$

Proof. There do not appear to be any shortcuts like Lemma 2.5 to the lower bound. However, the lists used to prove the same lower bound for FF [7, 8] are such that for any X satisfying the theorem's hypothesis, GXF will generate the same packing as FF (and for no X can GXF do any better). Thus $R[GXF, t] \geq R[FF, t] \geq 1 + 1/m$. The upper bound when X is as stated follows from Lemma 5.9 just as the upper bound in Theorem 2.6 followed from Lemma 2.4. ■

Remark. Thus for $t \leq 1/2$ and X the corresponding two-element set, we have $R[GXF, t] = R[FF, t]$. Although we cannot prove it, we conjecture that if $t > 1/2$ and $\{1/6, 1/3, 1/2\} \subseteq X$, then we also have $R[GXF, t] = R[FF, t] = 17/10$.

So far we have been considering on-line algorithms based on linear-time packing rules. Where preprocessing is possible, we might consider using the grouping idea to get an approximation to the DECREASING rule, whereby large pieces are generally made to come before small pieces, although the resulting list need not be strictly in decreasing order. Given a schedule of intervals X , our rule follows.

GROUPING-X RULE. L is ordered so that for all $x_i \in X$, if $\text{size}(a) > x_i$, $\text{size}(b) \leq x_i$ then $\text{rank}(a) < \text{rank}(b)$.

If X has k groups, a list of length n can be so grouped in about $n \cdot \log_2(k)$ comparisons. Moreover, if we are going to use the GROUP-X FIT packing rule, we would probably have to make many of those comparisons anyway, in the course of deciding which bin the pieces should go in. So, letting GROUP-X FIT GROUPED ($GXFG$) be the algorithm which preprocesses according to the GROUPING-X rule then packs according to GROUP-X FIT, we have that $GXFG$ probably takes little more time than GXF (which has *no* preprocessing), and yet can have better worst case behavior.

THEOREM 5.11. For $m = \lfloor 1/t \rfloor \geq 1$ and $\{1/(m+2), 1/(m+1), 1/m\} \subseteq X$,

$$R[GXFG, t] = 1 + 1/(m+1).$$

Proof. A general lower bound example for all possible X is given in Fig. 17. No matter what X is, the ϵ in the examples can be made small enough so that the GXF packing comes out as pictured. For the upper bound when X is as stated, let L be a list with $\text{Range}(\text{size}_L) \subset (0, 1/m]$ which is ordered in accordance with the GROUPING- X rule. Let PX be a GXF -packing of L using $GXF(L)$ bins. We shall prove that

$$(a) \quad L^* \geq [(m+1)/(m+2)] \cdot [GXF(L) - 1],$$

and the upper bound will follow as in Theorem 2.6. Divide the list into segments $L = L1 \cdot L2 \cdot L3$, where

$$\text{Range}(\text{size}_{L1}) \subset (1/(m+1), 1/m],$$

$$\text{Range}(\text{size}_{L2}) \subset (1/(m+2), 1/(m+1)], \quad \text{and}$$

$$\text{Range}(\text{size}_{L3}) \subset (0, 1/(m+2)].$$

This must be possible because $\{1/(m+2), 1/(m+1)\} \subseteq X$.

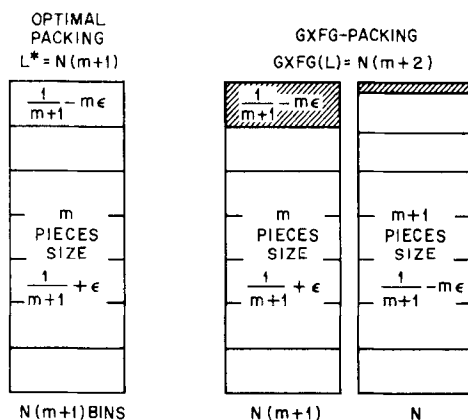


FIG. 17. Packings of list L for which $\text{Range}(\text{size}_L) \subset (0, 1/m]$ and

$$GXFG(L)/L^* = 1 + 1/(m+1).$$

Now if $b = \text{piece}_{PX}(\#PX, 1) \in \text{PIECES}(L3)$, then $\text{size}(b) \leq 1/(m+2)$, and when b was assigned, $\text{gap}_P(\#PX) = 1$. Thus by Lemma 5.7 and the fact that $1/(m+2) \in X$, we must have $\text{gap}_P(j) < 1/(m+2)$, $1 \leq j < \#PX$, so that

$$L^* \geq W(L) > [(m+1)/(m+2)] \cdot [GXF(L) - 1],$$

as desired.

Thus the only problem is if b is a piece from $L1$ or $L2$, that is if $GXF(L) = GXF(L12)$, where $L12 = L1 \cdot L2$. Thus, since $L^* \geq L12^*$, it will be sufficient to show that

$$(b) \quad L12^* \geq [(m+1)/(m+2)] \cdot [GXF(L12) - 1].$$

So let PX' be a GXF -packing of $L12$, using $GXF(L12)$ bins. Divide it into two segments as shown in Fig. 18. $PX1$ is the first $L12^*$ bins and $PX2$ is all the bins to the right of $PX1$. We may assume that $PX2$ is not vacuous, else (b) would be immediate.

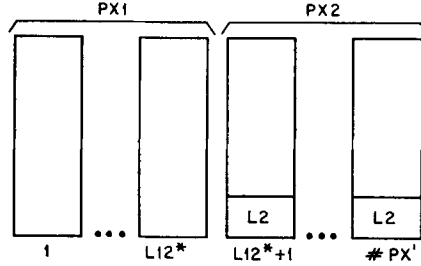


FIG. 18. Diagram for Theorem 5.11.

Now by size constraints we know that $L12^* \geq \lceil |\text{PIECES}(L1)|/m \rceil$. Let $j1 = \text{MAX}\{j: \text{piece}_{PX'}(j, 1) \in \text{PIECES}(L1)\}$. By Lemma 5.8 all BIN_j 's with $1 \leq j < j1$ must contain m pieces from $L1$, and so, $j1 \leq \lceil |\text{PIECES}(L1)|/m \rceil \leq L12^*$. Thus, since BIN_{j1} is the rightmost bin containing pieces from $L1$, all the pieces in $PX2$ must be from $L2$ and hence have size $\leq 1/(m+1)$.

Let us return our attention to $PX1$. By Lemma 5.8, since $\#PX1 < \#PX'$, $PX1$ must contain at least $(m)(\#PX1) = (m)(L12^*)$ pieces. But by size constraints $|\text{PIECES}(L12)| \leq (m+1)(L12^*)$. Thus the number of pieces in $PX2$ can be at most $L12^*$. Since $PX2$ can be considered a packing of a sublist of $L12$, all of whose pieces are $\leq 1/(m+1)$ in size, Lemma 5.8 tells us that all its bins except the last must contain at least $m+1$ pieces, and so $\#PX2 \leq \lceil L12^*/(m+1) \rceil$. Thus

$$\begin{aligned} GXF(L12) &= \#PX1 + \#PX2 \leq L12^* + \lceil L12^*/(m+1) \rceil \\ &\leq \lceil (m+2)/(m+1) \rceil L12^* + 1, \end{aligned}$$

and (b) follows. ■

Remark. Note that this is the one theorem which does not make a special case out of $t > 1/2$. For such t it yields $R[GXFG, t] = 1.5$. If we use an AF packing rule S instead of GXF along with the Grouping-X rule (algorithm SXG), we will get the same results for $t \leq 1/2$, but for $t > 1/2$ and $\{1/4, 1/3, 1/2\} \subseteq X$ we have the stronger result [7]:

$$R[SXG, t] = 4/3.$$

The bounds proved in this paper are summarized in Table I. The entries for algorithms involving grouping are all made under the assumption that X contains the numbers specified in the appropriate theorem.

TABLE I

Algorithms	Timing	$R[S, t], 1/2 < t \leq 1$	$R[S, t], m = \lfloor 1/t \rfloor \geq 2$
1. Worst fit	$O(n \log n)$	2.0	$1 + \frac{1}{1/t - 1}$
2. Next-1 fit	$O(n)$		
3. Next- k fit $k \geq 2$	$O(n)$	$\left[1.7 + \frac{3}{10k}, 2.0\right]$	
4. Group- X fit	$O(n)$	$[1.7, 2.0]$	$1 + \frac{1}{m}$
5. First fit	$O(n \log n)$		
6. Best fit	$O(n \log n)$	1.7	
7. $S \in AAF$	$\geq O(n \log n)$		
8. Group- X fit Grouped	$O(n)$	1.5	$1 + \frac{1}{m+1}$
9. S - X grouped $S \in AF$	$\geq O(n \log n)$	$\frac{4}{3} = 1.333...$	
10. First fit Decreasing	$O(n \log n)$	$\frac{11}{9} = 1.222...$	$\left[1 + \frac{1}{m+2} - \frac{2}{m(m+1)(m+2)}, 1 + \frac{1}{m+2}\right]$
11. Best fit Decreasing	$O(n \log n)$		
12. S decreasing $S \in AF$	$\geq O(n \log n)$	$\left[\frac{11}{9}, \frac{5}{4}\right]$	

CONCLUSION

This paper has suggested a variety of alternatives to the simple FIRST and BEST FIT heuristics and shown them to have similar worst case behavior. It is still an open question, however, whether there are any polynomial-time on-line algorithms with *better* worst case behavior than *FF*, or any off-line algorithms, perhaps not conforming to our two-part format, better than *FFD*.

Moreover, there is the whole problem of *average* case behavior. To be sure, our worst case analysis can have considerable practical significance by giving us insight into the mechanisms which cause worst case behavior and by suggesting alternative algorithms to cope with these difficulties. However, empirical studies with randomly generated lists [7] indicate that although comparisons between algorithms with respect to average case behavior reflect our worst case results, the actual behavior is considerably better. *FFD* for example uses an average of 2% excess bins rather than 22.2%. Moreover, there are some anomalies with respect to our linear-time algorithms, which though predictable by common sense, are not predicted by our worst-case results.

For instance, with the schedule $X = \{0, 1/3, 1/2, 1\}$, *GXF* has the same worst case behavior as *BF* on lists L with $\text{Range}(\text{size}_L) \subset (0, 1/2]$, but one would expect *GXF* to be worse in the average case because it has a greater tendency to leave large gaps in bins. And this was the case in our empirical studies, since *BF* required an average of 2% excess bins for such lists, while *GXF* required 25%, much closer to their mutual worst case bounds of 50%. This is not to imply that for *every* schedule X , *GROUP-X FIT* is worse than *BF*. Indeed, for the schedule of intervals

$$X(k) = \left\{0, \frac{1}{2^k}, \frac{2}{2^k}, \dots, \frac{2^k - 1}{2^k}, 1\right\}$$

our empirical results show $GX(k)F$ approaching *BF* in average case behavior as k increases, becoming practically indistinguishable from it for $k \geq 8$. This happens in spite of the fact that the worst case behavior for $GX(k)F$ is at least as bad as that for the original schedule X . Moreover, the average case behavior of $GX(k)FG$ approaches that for *BFD*, even though $GX(k)FG$'s worst-case behavior is at least as bad as that given in Theorem 5.11, and hence definitely worse than that for *BFD*.

Common sense tells us that these results are reasonable, but a rigorous analysis of such average case behavior, even if only for very simple distributions of pieces, would be very interesting.

ACKNOWLEDGMENTS

The author wishes to thank Professor M. J. Fischer and Professor A. R. Meyer for their interest and suggestions, especially the former for extensive editing help, and an anonymous referee for providing the result given here as Theorem 5.6.

REFERENCES

1. G. M. ADEL'SON-VEL'SKII AND YE. M. LANDIS, An algorithm for the organization of information, *Soviet Math. Dokl.* 3 (1962), 1259-1262.

2. A. R. BROWN, "Optimum Packing and Depletion," Macdonald and American Elsevier Inc., New York, 1971.
3. S. A. COOK, The Complexity of Theorem-Proving Procedures, Proceedings of the 3rd Annual ACM Symposium on the Theory of Computing, 1971.
4. A. DEMERS, private communication.
5. M. R. GAREY, R. L. GRAHAM, AND J. D. ULLMAN, Worst-Case Analysis of Memory Allocation Algorithms, Proceedings of the 4th Annual ACM Symposium on the Theory of Computing, 1972.
6. D. S. JOHNSON, Approximation Algorithms for Combinatorial Problems, Proceedings of the 5th Annual ACM Symposium on the Theory of Computing, 1973.
7. D. S. JOHNSON, Near-Optimal Allocation Algorithms, Ph.D. Dissertation, Massachusetts Institute of Technology, Cambridge, MA, June 1973.
8. D. S. JOHNSON, J. D. ULLMAN, A. DEMERS, R. L. GRAHAM, AND M. R. GAREY, in preparation.
9. R. M. KARP, Reducibility Among Combinatorial Problems, in "Complexity of Computer Computations" (R. E. Miller and J. W. Thatcher, Eds.), Plenum Press, New York, 1972.
10. J. NIEVERGELT AND E. M. REINGOLD, Binary Search Trees of Bounded Balance, Proceedings of the 4th Annual ACM Symposium on the Theory of Computing, 1972.
11. J. D. ULLMAN, The Performance of a Memory Allocation Algorithm, Technical Report No. 100, Princeton University, Princeton, NJ, 1971.